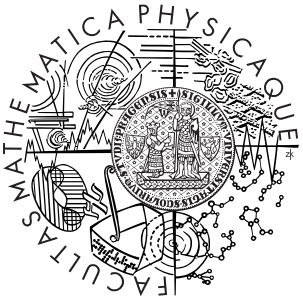
Univerzita Karlova v Praze

Matematicko-fyzikální fakulta

DIPLOMOVÁ PRÁCE



Bartłomiej Etenkowski

API pro ovládání robota v přirozeném jazyce

Ústav formální a aplikované lingvistiky

Vedoucí diplomové práce: Prof. RNDr. Jan Hajič, Dr.

Studijní program: informatika, matematická lingvistika

2011

Prohlašuji, že jsem svou diplomovou práci napsal samostatně a výhradně s použitím citovaných pramenů. Souhlasím se zapůjčováním práce a s jejím zveřejňováním.

Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorského zákona v platném znění, zejména skutečnost, že Univerzita Karlova v Praze má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona.

V Praze dne 8. prosince 2011 Bartłomiej Etenkowski

Obsah

1 Úvod **8**

2 Rešerše **12**

2.1 První systémy 13

2.2 Vyhledávání informací 16

2.3 Ovládání softwaru 18

2.4 Ovládání přístrojů 21

3 Nabaztag:tag **23**

3.1 Historie 24

3.2 Technická specifikace robota 26

3.3 Popis komunikačního protokolu 28

4 Obecný návrh **35**

4.1 Vrstvový model API 35

4.2 Mezijazyk 39

4.3 Základní datové typy 42

4.4 Služby 42

4.5 Ovladač králíka Nabaztag:tag 48

5 Použité technologie **50**

5.1 Java 50

5.2 Eclipse 3.5 a Eclipse 3.6 50

5.3 Spring Framework 3.0.5 51

5.4 Log4j 1.2.16 53

5.5 JUnit 56

5.6 Prolog 56

5.7 Epos 2.4.83 57

5.8 SoX 14.3.2 59

5.9 Rome 1.0 59

6 Implementace **60**

6.1 Hardwarové a systémové požadavky 60

6.2 Implementace jednotlivých vrstev 61

6.3 Logování a trasování 62

6.4 Internacionalizace 64

6.5 Debugování 65

6.6 Automatické testy 66

6.7 Vydání distribuce 66

6.8 Přehled komponent 67

6.8.1 Obecné API – `volkcore` 68

6.8.2 Aplikační framework – `volkapp` 70

6.8.3 Rozhraní pro formální gramatiky – `nlu_prolog` 72

6.8.4 HTTP server – `volkhs` 74

6.8.5 Ovladač králíka Nabaztag:tag – `volk` 75

6.8.6 Generování sdělení na základě vzorců – `nlpat` 85

6.8.7 Rozpoznání jazyka – `tool_langr` 91

7 Závěr **95**

7.1 Další možnosti vývoje 97

Literatura **99**

A Obsah přiloženého CD **102**

B Nastavení a použití robotického králíka Nabaztag:tag **104**

B.1 Konfigurace robota 104

B.2 Konfigurace serveru 106

B.3 Konfigurace logování 108

B.4 Spuštění serveru 108

B.5 Nahrávání zvuku 109

Název práce: API pro ovládání robota v přirozeném jazyce

Autor: Bartłomiej Etenkowski

Katedra (ústav): Ústav formální a aplikované lingvistiky

Vedoucí diplomové práce: Prof. RNDr. Jan Hajič, Dr.

e-mail vedoucího: hajic@ufal.mff.cuni.cz

Abstrakt: V předložené práci popisujeme API (obecného rozhraní) pro ovládání robota pomocí přirozeného jazyka a konkrétní implementaci tohoto API v programovacím jazyce Java na robotickém Wi-Fi králíkovi Nabaztag:tag. Součástí práce je rovněž návrh obecně použitelných lingvistických rozhraní (např. generování řeči z psaného textu, rozpoznávání jazyka). Popisujeme další možnosti vývoje API a návrh nových komponent.

Klíčová slova: NLP, API, Java, robot, Nabaztag

Title: API for natural language robot control

Author: Bartłomiej Etenkowski

Department: Institute of Formal and Applied Linguistics

Supervisor: Prof. RNDr. Jan Hajič, Dr.

Supervisor's e-mail address: hajic@ufal.mff.cuni.cz

Abstract: In the present work we describe the API for natural language robot control and an implementation of the API using Java programming language and the Nabaztag:tag – an ambient electronic device in the shape of a rabbit. We also describe the design of common linguistic interfaces (eg. text-to-speech

service, language recognition). Some further possibilities for the development of the API and the application are described in the present work too.
Keywords: NLP, API, Java, robot, Nabaztag

Kapitola 1

Úvod

Roboti, nebo nějaké jejich obdoby (např. *železní lidé*), se vyskytují v literatuře už mnoho let. Jedním z prvních byl Talus – železný člověk, který pomáhal hlavnímu hrdinovi anglického epického díla z konce 16. století. Od začátku 20. století, díky technickému pokroku, se literatura (a film) se zmiňuje o robotech stále častěji. Dokonce samotné slovo *robot* bylo převzato z literatury – poprvé ho použil Karel Čapek kolem roku 1920 v divadelní hře R. U. R (i když podle některých zdrojů byl vynálezcem slova jeho bratr Josef) a z češtiny se dostalo do mnoha jiných jazyků.

Skoro ve všech literárních dílech, ve kterých se vyskytují roboti nebo jiné inteligentní stroje, probíhá komunikace mezi nimi a lidmi (často i mezi roboty samotnými) v přirozeném jazyce. Takto komunikoval např. Arthegall s Talem nebo titulní hrdina Příběhů pilota Pirxe Stanisława Lema se zařízeními, která ovládala vesmírné lodě. V angličtině komunikoval taky nadporučík Dat – android, člen posádky vesmírné lodě USS Enterprise D (problematické byly pro něj vlastně jen vtipy).

Struktura práce

Po tomto úvodu následuje krátký exkurs do problematiky ovládání počítačů a jiných zařízení hlasem nebo psanými pokyny (kapitola 2). Celá kapitola 3 je věnována robotickému králíkovi Nabaztag:tag. Poté popisujeme obecný návrh (kapitola 4) a konkrétní implementaci (kapitola 6). Kapitoly 4 a 6 obsahují výsledky naší vlastní práce. Použité technologie, software a komponenty třetích stran jsou popsány v kapitole 5. Na závěr, v kapitole 7, naznačujeme další možnosti vývoje programu.

Obsah práce

Naším cílem bylo vytvořit obecný návrh API pro ovládání robota v přirozeném jazyce a ukázkovou implementaci na robotickém Wi-Fi králíkovi Nabaztag:tag.

1. Definice API mezi agentem (robotickým králíkem Nabaztag:tag) a ovladačem (komunikačním serverem) v následujících oblastech:
   - komunikace mezi serverem a robotem,
   - komunikace mezi uživatelem a serverem včetně možnosti zpracování přirozeného jazyka,
   - komunikace se službami třetích stran,
   - zpracování získané informace (generování hlasu, analýza vstupu od uživatele).
2. Implementace serveru pro komunikaci s robotem a zpracování vstupu a výstupu z robota:

- Komunikační protokoly.
- Vstup (robot → server):
  - záznam hlasového vstupu (digitální audio).
- Výstup (server → robot):
  - poslání příkazu pro pohyb,
  - poslání příkazu pro světelné efekty,
  - poslání hlasového signálu (digitální audio) – komunikace s uživatelem,
  - vstup od uživatele (nastavení, zápis zprávy k poslání a přečtení robotem).
- Komunikace se službami třetích stran:
  - sběr informací z internetu (RSS).
- Zpracování vstupu ze všech zdrojů z hlediska přirozeného jazyka:
  - jednoduchý interpret příkazů pro robota v českém jazyce,
  - roznoznávání jazyka textu,
  - vytváření zpráv v přirozeném jazyce na základě vzorců,
  - generování hlasového výstupu z psaného textu.

3. Příkladové aplikace:
   - rozpoznávání a provádění jednoduchých příkazů v češtině
   - čtení RSS zpráv

Použité konvence

V této práci jsme použili následující konvence:

- Názvy javovských balíků, typů, metod, vlastností a členských proměnných typů jsou vysázeny neproporcionálním písmem, např. `balik.Typ`.
- Víceřádkové ukázky kódu jsou vysázeny neproporcionálním písmem a jsou navíc orámovány, např.:

```
package balik;
/** Nějaká jednoduchá třída. */
class Example {
  /** Metoda, která nedělá vůbec nic. */
  public void doNothing() {
  }
}
```

Příklad 1.1: Ukázka kódu v programovacím jazyce Java.

- Nové pojmy v textu jsou vysázeny kurzívou, např.: „tento objekt se bude jmenovat *superobjekt*".

Kapitola 2

Rešerše

První systémy, které umožňovaly rozhovory s počítačem nebo ovládání softwaru pomocí přirozeného jazyka, začaly vznikat v 60. letech 20. století. Na začátku se jednalo o jednoduché dialogové systémy (např. ELIZA[17]), které nedisponovaly žádnými znalostmi a jejich fungování bylo založeno čistě na formálních lingvistických transformacích. Postupem času vznikaly stále komplikovanější systémy, které umožňovaly např. ovládání robota (byť jen virtuálního, jako např. SHRDLU [20]) a dotazování pomocí přirozeného jazyka. Stále se však ještě nejednalo o použitelné v praxi systémy.

S rozvojem vědy a techniky vznikly nástroje, které kromě vědeckého uplatnění našly i uplatnění praktické. Jednalo se např. o systémy vyhledávání informací v přirozených jazycích (např. Wolfram|Alpha). Díky pokroku v oblasti generování a rozpoznávání mluvené řeči se otevřely nové možnosti ovládání softwaru a zařízení. Vznikají nástroje, které umožňují zrakově postiženým uživatelům pracovat s počítačem nebo prostě ovládat software pomocí hlasových příkazů (Opera, MS Windows, Mac OS X). Hlasovými příkazy v přirozených jazycích lze

ale ovládat nejen software na osobních počítačích. Hlasové ovládání se uplatňuje např. v částečně automatizovaných call centrech a dokonce i v moderních stíhacích letounech.

V této kapitole na základě nejdůležitějších a nejzajímavějších projektů stručně popisujeme dějiny a současný stav výzkumu v oblasti použití přirozeného jazyka pro ovládání softwaru a robotů.

2.1 První systémy

ELIZA

ELIZA byla jedním z prvních dialogových systémů, které umožňují konverzaci člověka s počítačem. Program simuluje psychoanalytika při rozhovoru s pacientem (uživatelem).

ELIZA neobsahuje žádnou komponentu, která odpovídá za sémantickou analýzu, a nesnaží se žádným způsobem porozumět rozhovoru [17]. Odpovědi programu jsou generovány pomocí *transformací* vět zadaných uživatelem. ELIZA v zadané větě vyhledává *klíčová slova*, z nichž nejdůležitější vybírá a na základě těchto klíčových slov a transformačních pravidel vytváří odpověď. Program si věty zadané uživatelem ukládá do paměti. Pokud uživatel zadá větu, která neobsahuje klíčové slovo nebo se na ni nedá použít žádné transformační pravidlo, ELIZA se buď vrátí k jednomu z předchozích témat (to může udělat, protože si vstupy ukládá), nebo položí nějakou obecnou otázku (např. *In what way?*). Příklad 2.1 znázorňuje, jak probíhá skutečný *rozhovor* uživatele s Elizou.

U (uživatel): Men are all alike.
E (ELIZA): In what way?

U: They're always bugging us about something specific or other.

E: Can you think of a specific example?

U: Well, my boyfriend made me come here.

E: Your boyfriend made you come here?

U: He says I'm depressed much of the time.

E: I am sorry to hear you are depressed.

U: It's true. I'm unhappy.

E: Do you think coming here will help you not to be unhappy?

U: I need some help, that much seems certain.

E: What would it mean to you if you got some help?

U: Perhaps I could learn to get along with my mother.

E: Tell me more about your family.

U: My mother takes care of me.

E: Who else in your family takes care of you?

U: My father.

E: Your father.

U: You are like my father in some ways.

E: What resemblance do you see?

U: You are not very aggressive but I think you don't want me to notice that.

E: What makes you think I am not very aggressive?

U: You don't argue with me.

E: Why do you think I don't argue with you?

U: You are afraid of me.

E: Does it please you to believe I am afraid of you?

U: My father is afraid of everybody.

E: What else comes to mind when you think of your father?

U: Bullies.

E: Does that have anything to do with the fact that your boyfriend made you come here?

Příklad 2.1: Příklad skutečného *rozhovoru* uživatele s programem ELIZA[17].

SHRDLU

SHRDLU je systém, který pomocí příkazů v přirozeném jazyce umožňuje ovládat virtuálního robota. Vznikl v letech 1968–1970 v rámci disertace Terryho Winograda na MIT [20]. Winogradovým cílem bylo vytvoření programu, který, na rozdíl od systému ELIZA, dokáže přirozenému jazyku skutečně porozumět. SHRDLU umožňuje *rozhovor* v anglickém jazyce s virtuálním robotem, který manipuluje s kostkami ze stavebnicemi různých barev, velikostí a tvarů. SHRDLU disponuje bází znalostí, kam ukládá pozice jednotlivých dílků stavebnice. Ta obsahuje rovněž základní fyzikální zákony (žádný dílek nemůže viset ve vzduchu, jehlan lze položit na krychli, ale naopak nikoli atp.). Na základě příkazů v angličtině SHRDLU svou bázi znalostí modifikuje. Kromě toho umí odpovídat na otázky týkající se aktuální situace ve svém okolí. Příklad 2.2 ukazuje záznam skutečného *rozhovoru* mezi programem a uživatelem.

U (uživatel): Pick up a big red block.

S (SHRDLU): Ok. (does it)

U: Grasp the pyramid.

S: I don't understand which pyramid you mean.

U: Find a block which is taller than the one you are holding and put it into the box.

S: By "it", I assume you mean the block which is taller than the one I am holding.

U: Ok. (does it)

S: What does the box contain?

U: The blue pyramid and the blue block.

S: What is the pyramid supported by?

U: The box.

S: How many blocks are not in the box?

U: Four of them.

Příklad 2.2: Příklad *rozhovoru* se systémem SHRDLU (`http://hci.stanford.edu/~winograd/shrdlu/`). V *rozhovoru* jsou vidět zároveň příkazy, jak i různé dotazy týkající se aktuální situace na stole a odpovědi na tyto dotazy. Originál *rozhovoru* je zapsán čistě velkými písmeny.

2.2 Vyhledávání informací

Vyhledávání v přirozeném jazyce, na rozdíl od klasického vyhledávání pomocí klíčových slov (jaké používá mj. populární internetový vyhledávač Google), umožňuje nacházení odpovědí na konkrétní otázky místo stránek, na kterých se prostě zadaná slova vyskytují. Díky tomu taky mohou najít požadované informace např. osoby, které kvůli nedostatku zkušeností s moderní výpočetní technikou nebo třeba psychické poruše nejsou schopny sestavit správný dotaz.

Wolfram|Alpha

Wolfram|Alpha umožňuje vyhledávání konkrétních informací na základě nestrukturovaných dotazů v angličtině (jiné jazyky nejsou zatím podporovány). Dlouhodobým cílem projektu je zpřístupnit všechny znalosti na jednom místě[1] a umožnit dotazování v přirozeném jazyce. Několik příkladů skutečných dotazů a odpovědí na ně ukazuje tabulka 2.1.

Dotaz	Výsledek
warsaw prague	Porovnání obou měst (počet obyvatel, plocha, časový rozdíl, počasí, nadmořská výška), vzdálenost a délka letu mezi městy
distance between warsaw and prague	Vzdálenost mezi městy v různých jednotkách, délka letu v rychlosti průměrného dopravního letadla, v rychlosti zvuku a světla.
distance from warsaw to prague	
how long does it take to get from warsaw to prague	Podobný výsledek jako výše, akorát na prvním místě je čas.
who was born in warsaw	Seznam známých lidí narozených ve Varšavě.
who was president after masaryk	Informace o prezidentu Václavu Klausovi. Vyhledávač nicméně upozorní uživatele na to, že neporozuměl vstupu a odpověď generuje na základě své nejbližší interpretace: *president*.

Tabulka 2.1: Wolfram|Alpha – příklady dotazů a popisy odpovědí na ně.

2.3 Ovládání softwaru

Windows Speech Recognition

Modul pro rozpoznávání mluvené řeči (*Windows Speech Recognition*) je součástí desktopových operačních systémů Microsoft Windows od verze XP. V systému Microsoft Windows 7 je v současné době dostupný pro angličtinu, francouzštinu, španělštinu, němčinu, japonštinu a čínštinu. Podle informací na oficiálních webových stránkách Microsoftu [19] modul umožňuje ve výše zmíněných jazycích diktovat text a také ovládat software na základní úrovni, např. spustit program, kliknout na vybraný prvek, scrollovat, simulovat stisknutí tlačítka na klávesnici atp.

Opera

Internetový prohlížeč Opera od verze 8.0 (viz [7]) umožňuje ovládání hlasem, které je oficiálně dostupné pouze pod operačními systémy Windows 2000 a Windows XP a funguje jen v angličtině.

Ubiquity

Ubiquity je doplněk pro internetový prohlížeč Firefox. Byl vyvíjen v rámci Mozilla Labs v letech 2008–2009. Cílem tohoto projektu bylo podle [16] zjištění, zda použití příkazového řádku, který dokáže zpracovávat příkazy v přirozeném jazyce, může zrychlit a zjednodušit prohlížení internetových stránek. Zkoumají se dvě hlavní oblasti:

lingvistická rozhraní *Linguistic interfaces.* Může být v praxi použitelné rozhraní, kam uživatel napíše, co chce provést? Lze ho naprogramovat pro jiné

jazyky než angličtinu?

slovesa na webu *Verbifying the Web.* Velké množství stránek na Internetu zpřístupňuje uživatelům *služby*, pomocí kterých lze např. překládat, vyhledávat informace, přidávat příspěvky na fóra atp. Ubiquity umožňuje zacházet s takovými stránkami jako s nástroji, které lze volat pomocí *sloves*. Lze tyto služby používat nikoli jako obyčejné webové stránky, nýbrž jako nástroje ovládané pomocí obecného lingvistického rozhraní?

AppleScript

AppleScript je objektově orientovaný skriptovací jazyk, podporovaný mnoha aplikacemi pod operačním systémem Mac OS X[11]. Jeho syntaxe má svůj základ v přirozeném jazyce (angličtině) a je na základní úrovni snadno čitelná i pro osoby, které v tomto jazyce nikdy neprogramovaly. Příklad 2.3 obsahuje ukázku zdrojového kódu v jazyce AppleScript.

```
tell application "Safari"
  set promptText to "Select file containing URLs"
  set inputFile to choose file with prompt promptText
  set urls to paragraphs of (read inputFile) as list
  repeat with loc in urls
    open location loc
  end repeat
end tell
```

Příklad 2.3: Ukázka kódu v jazyce AppleScript. Skript po spuštění zobrazí uživateli dialogové okno pro výběr vstupního souboru. Předpokládá se, že každý řádek obsahuje správnou URL adresu. Skript tyto adresy načte a každou otevře v internetovém prohlížeči Safari.

Programovací jazyk AppleScript obsahuje mnoho rysů, díky kterým jsou skripty velmi podobné textům v přírozeném jazyce (angličtině). Níže uvádíme některé z nich.

1. Klíčové slovo `the`. Je nepovinné a bylo zavedeno jen proto, aby se skript více podobal textu v přirozeném jazyce. Následující příkazy:

   - `version of application "Finder"`
   - `the version of the application "Finder"`

   jsou si ekvivalentní.

2. Klíčová slova `of` a `in`. Slouží pro přístup k objektům a vlastnostem kontejnerů a v tomto kontextu jsou si rovnocenné (programátor si může mezi nimi vybrat podle vlastního uvážení, ve většině případů bude zvolena varianta, která se více hodí z hlediska přirozeného jazyka). Lze tedy napsat:

   - `the version of the application "Finder"` nebo
   - `the version in the application "Finder"`

   Přestože programátor zřejmě zvolí první variantu, výsledek volání obou příkazů bude stejný.

3. Přípona `'s`. Může být použita místo klíčového slova `of` nebo `in` pro přístup k objektům nebo vlastnostem kontejnerů. Např. následující příkazy:

   - `the name of the first item of the first disk`
   - `first disk's first item's name`
   - `first item's name of the first disk`

jsou si rovnocenné a je věcí programátora, kterou variantu zvolí. Je-li tento příkaz adresován aplikaci *Finder*, všechny výše uvedené varianty vrátí název prvního objektu (složky nebo souboru) na prvním disku v počítači.

4. Zájmena `me`, `my`, `it` a `its`. Klíčové slovo `me` odkazuje na právě spuštěný skript, `it` na výchozí cíl volání.

2.4 Ovládání přístrojů

Mobilní telefony

Některé mobilní telefony (např. Nokia E52, Sony Ericsson K800i) umožňují kromě dotykového ovládání na základní úrovni i ovládání hlasové (např. výběr telefonního čísla, odeslání SMS zprávy, příjem hovoru). Je určeno hlavně pro použití s handsfree sadami do auta a podle výrobců zvyšuje bezpečnost, když řidič telefonuje za jízdy.

Stíhací letouny

Některé moderní stíhací letouny umožňují pilotům ovládat hlasem vybrané přístroje, takže se pilot může plně soustředit na řízení letadla. Např. stíhací letouny Eurofighter Typhoon disponují DVI (*Direct Voice Input*) modulem[4]. Pomocí DVI lze hlasovými příkazy zadávat parametry letu či ovládat některé palubní systémy. Nejedná se nicméně o prvky kritické z hlediska bezpečnosti letadla. Hlasem lze ovládat mj.:

- nastavení displejů a výběr položek, které mají na nich být zobrazeny
- výběr požadovaného rádiového kanálu

- nastavení trasy
- výběr cíle útoku
- přiřazování cílů útoku jednotlivým letadlům v skupině

Nabaztag:tag

Ovládání pomocí přirozeného jazyka umožňuje i robotický králík Nabaztag:tag (stejně jako jeho předchůdce, Nabaztag, a následník, Karrotz). Robotovi Nabaztag:tag věnujeme celou kapitolu 3.

Kapitola 3

Nabaztag:tag

Nabaztag:tag je hračka – robot ve tvaru králíka (viz obrázek 3.1). Robot je vybaven Wi-Fi kartou, díky níž se dokáže připojit k počítačové síti a komunikovat se serverem. Interakci s uživatelem zajišťují především reproduktory a mikrofon. Králík je navíc vybaven pohyblivýma ušima a pěti barevnými světýlky. Na hlavě je umístěno tlačítko.

Samotný robot, bez připojení k síti, neposkytuje žádnou zajímavou funkcionalitu. Až po připojení k ovládacímu serveru může dojít k načtení bootcodu (operačního systému zařízení). Potom robot tvoří rozhraní mezi uživatelem a ovládacím serverem, který zpracovává uživatelský vstup a generuje výstup. Může se jednat např. o projev v přirozeném jazyce nebo o choreografii – posloupnost příkazů, které řídí mj. polohu uší a barvu jednotlivých světel. Pro komunikaci mezi roboty a serverem se používají protokoly založené na standardech HTTP a XMPP. Popis používaného v této práci komunikačního protokolu, jehož základem je norma HTTP, se nachází v kapitole 3.3 na stránce 28.



Obrázek 3.1: Robotický králík Nabaztag:tag. Zdroj fotografie: `http://pl.wikipedia.org/w/index.php?title=Plik:Nabaztagtag_large.jpg&filetimestamp=20090129180541`.

3.1 Historie[1]

Robotického králíka Nabaztag:tag, stejně jako jeho předchůdce, Nabaztaga, vyráběla francouzská společnost Violet. Ta také provozovala jediný oficiální server. Roboti se serverem původně komunikovali pomocí HTTP protokolu, nicméně kvůli velkému zatížení serverů si uživatelé často stěžovali na velmi dlouhou dobu odezvy (dokonce v řádu minut). V březnu 2008 byl HTTP protokol při spojení s roboty nahrazen XMPP protokolem a doba odezvy se výrazně zmenšila, servery byly nicméně velmi nestabilní a docházelo k dlouhodobým výpadkům poskytovaných služeb.

V roce 2009 ohlásil výrobce robota úpadek a byl převzat společností Minds-

[1] Tato kapitola je založena na [18].

cape, známá především jako vydavatel vzdělávacích počítačových her. V červenci 2011 byla podpora dvou prvních verzí robota Nabaztag (Nabaztag a Nabaztag:tag) pro nerentabilitu ukončena a byl zveřejněn zdrojový kód operačního systému zařízení a serverů pro obě verze. V současné době je oficiálně podporována pouze třetí verze robota – Karotz.

Nabaztag:tag připojený k serverům `violet.net` umožňoval mj. přehrávání internetového rádia, předpovědí počasí, čtení e-mailů atp. Konkrétní chování záleželo na tom, které aplikace byly na uživatelském účtu robota aktivovány (pro přístup k serveru musel mít králík aktivní účet, identifikován byl pomocí svého sériového čísla). Zprávy, choreografie atp. se mohly aktivovat na základě pravidelných událostí (např. každou hodinu), vnějších událostí (přijetí e-mailu) nebo na základě událostí generovaných uživatelem (např. nahrání zvuku, který byl serverem rozpoznáván a interpretován).

Roboty bylo možné ovládat pomocí příkazů v přirozeném jazyce. Podpora v první fázi zahrnovala francouzštinu a angličtinu, v roce 2007 se rozšířila o němčinu, italštinu, španělštinu a holandštinu. Generování řeči na základě psaných textů (např. čtení zpráv) fungovalo i pro portugalštinu, dánštinu, finštinu, islandštinu, švédštinu, norštinu, turečtinu a polštinu. Zpracování uživatelských příkazů v přirozeném jazyce a generování zpráv probíhalo pouze na serveru, samotné zařízení umělo pouze nahrát zvuk a poslat ho na server, resp. vygenerovaný zvuk ze serveru stáhnout.

Velký zájem o internetové králíky a služby s nimi spojené způsobil, že společnost Violet začala vyrábět další zařízení, která dokázala s Nabaztagy spolupracovat. Mezi ně patřily:

mir:ror Čtečka RFID kódů; připojuje se k počítači pomocí USB portu. Reaguje

na přítomnost RFID čipů (ztamp:s, nano:ztag) a spouští na počítači akce přiřazené k jednotlivým čipům (např. otevření internetové stránky).

ztamp:s Barevné etikety se zabudovanými RFID čipy. Ty je možné přilepovat na různé předměty. Když se RFID čip dostane do bezprostřední blízkosti králíka Nabaztag:tagnebo zařízení mir:ror, provede robot akci, která je s ním spojena.

nano:ztag Menší barevná verze králíka. Uvnitř je zabudován čip ztamp:s, což umožňuje interakci s většími roboty Nabaztag:tag nebo se čtečkami mir:ror.

Výrobce zpřístupňoval API, které umožňovalo nezávislým poskytovatelům vytvářet a zpřístupňovat služby. Komunikace s králíky přitom probíhala prostřednictvím webových serverů výrobce.

3.2 Technická specifikace robota

- výška: 16 cm bez uší/23 cm s ušima
- průměr: 13,5 cm
- váha: 1 kg
- výměnné pohyblivé magnetické uši
- napájení: 110-230 V
- Wi-Fi karta: 802.11 b/g
- podpora šifrování: WEP a WPA
- dva reproduktory 2 W



Obrázek 3.2: Čtečka mir:ror, dva minikrálíci nano:ztag a tři etikety ztamp:s. Zdroj fotografie: `http://fr.wikipedia.org/wiki/Fichier:A-Mirror-2-Nanoztag-and-3-Ztamps.jpg`.

- audiovýstup jack 3.5 mm pro externí reproduktory nebo sluchátka
- ovládání hlasitosti, možnost úplně vypnout reproduktory
- mikrofon
- čtečka RFID, ISO14443 Type-B
- pět barevných diod – jedna na nose, tři na břichu a jedna na spodní straně
- multifunkční tlačítko na hlavě mezi ušima

3.3 Popis komunikačního protokolu

Nabaztag:tag se po spuštění připojí k serveru uvedenému v konfiguraci (viz příloha B.1 na stránce 104) a stáhne svůj operační systém (bootcode). V tomto kroku se vždy použije protokol HTTP. V dalších krocích robot komunikuje se serverem pomocí protokolu HTTP nebo XMPP – záleží na konkrétní verzi bootcode. V této práci používáme starší verzi, která podporuje komunikaci založenou na protokolu HTTP. Příslušný bootcode (soubor `bootcode.bin`) je součástí knihovny `volk` (viz kapitola 6.8.5) – tím je splněna jedna z podmínek nezávislosti naší aplikace na externích poskytovatelích služeb.

Protože zdrojový kód ovládacího serveru a operačního systému králíka byl zveřejněn až poté, co skončila oficiální podpora výrobce pro roboty Nabaztag a Nabaztag:tag, v červenci 2011, nebyla v době psaní této práce dostupná žádná oficiální specifikace komunikačního protokolu. Vycházeli jsme tedy z dokumentace k nezávislému serveru jNabServer[9], která vznikla díky reverse-engineeringu. Protože se nejedná o oficiální dokument výrobce, může tato specifikace obsahovat malé nepřesnosti. Níže popsaná komunikace je založena na protokolu HTTP 1.0.

Dotazy (Nabaztag:tag → server)

Níže uvádíme popis jednotlivých dotazů, které Nabaztag:tag odesílá na server. Ve všech příkladech je třeba nahradit řetězec `%s` skutečnou IP adresou serveru v lokální Wi-Fi síti. Pořadí jednotlivých volání a odpovědí je znázorněno na sekvenčním diagramu (obrázek 3.3 na stránce 34).

Téměř vždy se používá metoda `GET`; výjimkou je dotaz, pomocí kterého Nabaztag:tag posílá na server natočený zvukový záznam, v tom případě tak učiní

`POST`.

`GET http://%s/vl/bc.jsp` Tento dotaz králík odešle hned po svém spuštění a nalezení Wi-Fi sítě na server uvedený v konfiguraci. V odpovědi očekává bootcode.

`GET http://%s/vl/locate.jsp` Dotaz je volán po načtení správného bootcodu. Odpověď by měla obsahovat dvě IP adresy – jednu pro odesílání běžných dotazů, druhou pro odesílání natočeného zvuku. V této práci používáme pro oba účely stejný server. V případě serverů nezávislých na výrobci jsou typicky obě adresy shodné s adresou uvedenou v konfiguraci králíka. Díky tomuto dotazu lze roboty spravovat přes různé servery bez změny jejich konfigurace – je tedy nezbytný pro správu velkého množství robotů se stejným nastavením.

V tomto kroku se naposledy používá adresa nastavená v konfiguraci. Potom již králík komunikuje s adresami, které získá jako odpověď na tento dotaz.

`GET http://%s/vl/p4.jsp` Ping – tento dotaz zařízení odesílá v pravidelných intervalech (výchozí interval je 10 vteřin, interval lze za běhu robota změnit). Odpověď by měla obsahovat seznam příkazů, které má robot aktuálně vykonat (podrobný popis příkazů se nachází na stránce 30).

Králík pokračuje v odesílání požadavků, i když dojde k výpadku komunikace, způsobenému např. výpadkem serveru nebo nedostupností Wi-Fi sítě. Aby králík znovu načetl bootcode a adresy serverů, se kterými má komunikovat, je třeba provést restart zařízení – buď fyzický (odpojit králíka od elektřiny a zase připojit) nebo softwarový (speciálním příkazem).

`POST http://%s/vl/record.jsp` Tento dotaz zavolá robot po natočení zvukového záznamu. Jeho datová (`POST`) část obsahuje natočený zvuk zakódovaný ve formátu ADPCM2. Dotaz může být robotem pro každý zvukový záznam zavolán několikrát.

`GET http://%s/vl/rfid.jsp` Tento dotaz přichází v úvahu, když robot detekuje přítomnost RFID čipu. Metody pro práci s RFID čipy nebyly v této práci implementovány.

Příkazy (server → Nabaztag:tag)

Odpověď na dotaz `GET http://%s/vl/p4.jsp` typicky obsahuje balík příkazů pro robota. Ten sestává z pole bajtů, které vždy začíná znakem `0x7f` a končí sekvencí dvou bajtů `0xff 0x0a`. Uvnitř balíku se může nacházet libovolný počet příkazů. Každý příkaz se skládá ze tří částí:

1. Typ příkazu (vždy 1 bajt).
2. Délka datové části (vždy 3 bajty). Používá se kódování big-endian.
3. Datová část (proměnná délka). Nevyskytuje se u všech typů příkazů (tj. délka datové části může být nulová). Obsahuje data specifická pro daný typ příkazu.

Níže popisujeme jednotlivé příkazy. V závorkách u názvu uvádíme hodnotu bajtu označujícího typ.

Nastavení intervalu mezi ping dotazy (0x03) Pro tento typ příkazu datová část obsahuje vždy 1 bajt, který určuje, jak často (v sekundách) má robot odesílat na server ping dotazy.

Softwarový restart (0x09) Po přijetí toho příkazu Nabaztag:tag znovu odešle dotaz na bootcode, zeptá se na adresy serverů atd. Frekvence odesílání ping dotazu se přenastaví na výchozí hodnotu. Pokud balík s tímto příkazem obsahuje nějaké další příkazy, králík je ignoruje.

Skript (0x0A) je zvláštním typem příkazu. Dostupný skriptovací jazyk je velmi jednoduchý a umožňuje pouze volat předdefinované procedury. Jednotlivá volání jsou od sebe oddělena pomocí znaku nového řádku (`0x0D`). Některé procedury vyžadují parametr – ten se od jejich názvu odděluje jednou mezerou: `metoda parametr`.

Celý skript je zakódován následujícím způsobem:

$$E[i] = (T[i] - 47) \cdot (1 + 2 \cdot E[i-1]) \tag{3.1}$$

kde $E[i]$ je kód ASCII i-tého znaku zakódovaného textu, $T[i]$ je kód ASCII i-tého znak nekódovaného textu, $E[0] = -1$ (indexování textu od 1).

Dostupné jsou následující procedury:

`ID` Identifikátor skriptu. Každý skript musí obsahovat volání této procedury, jinak na něj králík nebude reagovat.

`CL` Rozsvícení světýlka. Procedura vyžaduje parametr v následujícím formátu: `0xLLRRGGBBh`, kde `LL` je číslo diody, a `RRGGBB` je definice požadované barvy v RGB modelu.

`PL` Nastavení palety barev pro choreografie. Parametr – id palety.

`CH` Přehrávání choreografie. Parametr – URL choreografie.

`ST` Přehrávání streamu, např. internetového rádia. Parametr – URL streamu.

`MU, MC` Přehrávání zvukového souboru (oba názvy označují tutéž proceduru). Robot nejprve celý soubor stáhne. Parametr – url souboru.

`MW` Očekávání na vykonání dříve volaných procedur. Metoda nepřijímá žádné parametry.

Choreografie

Choreografie jsou binární soubory, které obsahují posloupnosti příkazů ovládajících pohyby uší, rozsvěcení diod na králíkovi a přehrávání MIDI zvuků. Každý soubor začíná čtyřbajtovou hlavičkou s informací o délce datové části a končí čtyřmi nulovými bajty (viz příklad 3.1).

Datová část se skládá z posloupností bajtů, které reprezentují jednotlivé příkazy.

1. bajt určuje délku čekání na provedení příkazu po **začátku** provádění předchozího příkazu.

2. bajt určuje typ příkazu.

3. a následující bajty jsou parametry příkazu.

Dále prezentujeme seznam podporovaných příkazů. V závorkách uvádíme syntax příkazu – jeho typ (první bajt) a parametry (ostatní bajty) v hexadecimálním zápisu.

Nastavení tempa (`01TT`) Nastavení rychlosti přehrávání choreografie. Parametr `TT` určuje novou rychlost. Výchozí hodnota je `0x0a`.

Rozsvícení světla (`07LLRRGGBB0000`) Rozsvícení zvoleného světýlka. Parametr `LL` je číslo `0x00` až `0x04`, které určuje, které světlo se má rozsvítit. Následují tři bajty, které určují červenou, zelenou a modrou složku výsledné barvy. Na konci jsou vždy dva nulové bajty.

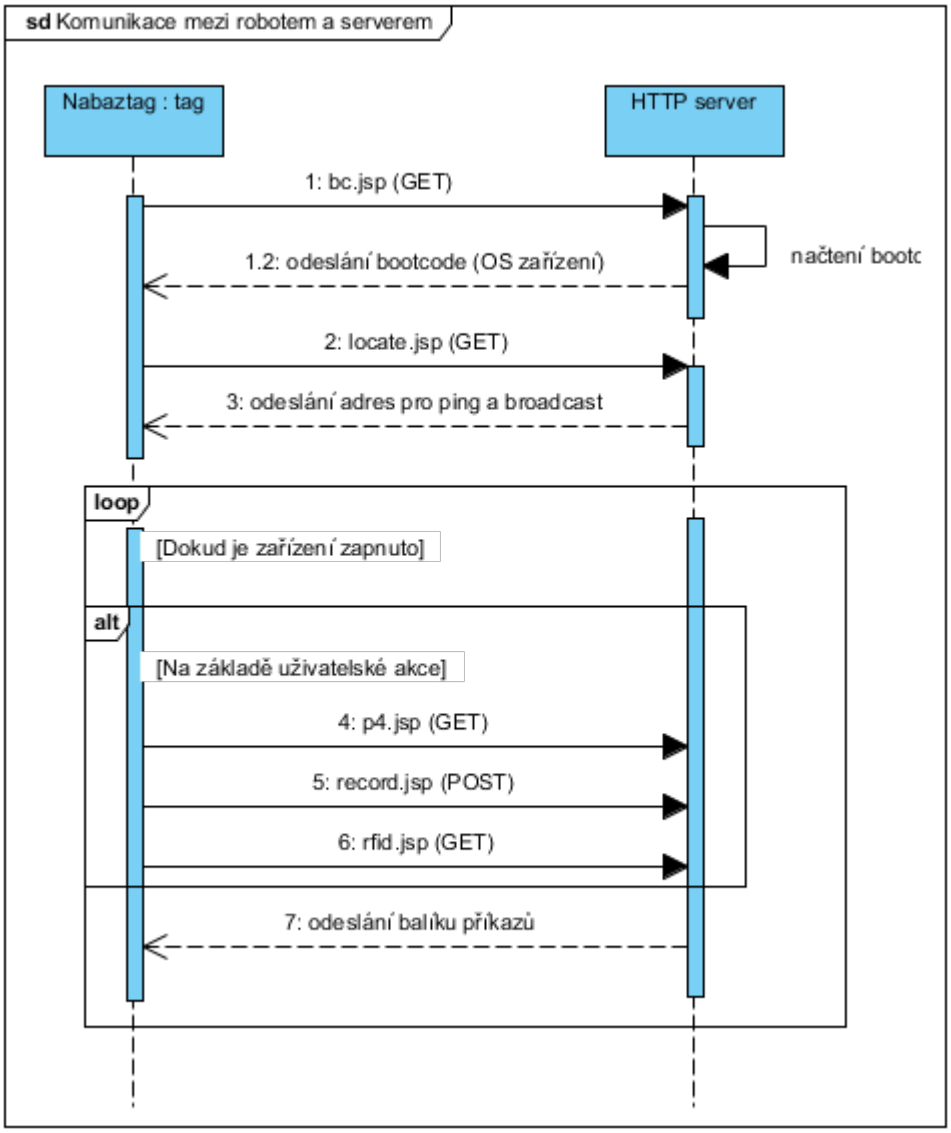
Nastavení absolutní pozice ucha (`08EEPPDD`) Parametr `EE` určuje, kterého ucha se příkaz bude týkat (`0x00` – pravé, `0x01` – levé). Pomocí následujícího parametru se nastavuje požadovaná pozice ucha (číslo v rozsahu `0x00` až `0x12`, `0x00` značí vzpřímenou pozici). Pomocí posledního parametru lze nastavit směr, ve kterém se má ucho naklopit (`0x00` – dopředu, `0x01` – dozadu).

Přehrání náhodného MIDI zvuku (`10`) Přehrání náhodně zvoleného krátkého zvuku. Tento příkaz nevyžaduje žádné parametry.

Změna pozice ucha (`11EESS`) Tento příkaz přijímá dva parametry – označení ucha a počet kroků, o kolik se má ucho otočit dopředu.

```
1 00 00 00 22
2 00 01 30
3 00 10
4 00 00 ff 00 00 00 00
5 01 01 00 ff 00 00 00
6 01 02 00 00 ff 00 00
7 01 08 00 00 00
8 01 08 01 00 01
```

Příklad 3.1: Příklad choreografie. Dvojice znaků zde odpovídají bajtům v hexadecimálním zápisu. Pro zlepšení čitelnosti byly přidány bílé znaky a číslování řádků.



Obrázek 3.3: Schéma komunikace mezi robotem a HTTP serverem.

Kapitola 4

Obecný návrh

API (*Application Programming Interface*) je běžné encyklopedické definice [3] sada standardizovaných požadavků, které umožňují vzájemnou komunikaci různých počítačových systémů. API slouží přístupu k zařízením za účelem získávání dat nebo ovládání, a to standardizovaným způsobem. Nemusí se přitom jednat o přímé připojení pomocí kabelu, stačí např. připojení pomocí Wi-Fi sítě, jako u robotického králíka Nabaztag:tag. API může být do velké míry nezávislé na konkrétním typu zařízení.

Tato kapitola obsahuje obecný popis návrhu API pro komunikaci s robotem v přirozeném jazyce. Kapitola 6 představuje ukázkovou implementaci a její technické detaily.

4.1 Vrstvový model API

Koncepce vrstev je obecně aplikovatelný model, který umožňuje rozdělit složitý systém na jednodušší a obecnější prvky. V ideálním případě by měly spolu ko-

munikovat pouze sousední vrstvy a používat přitom výhradně veřejná rozhraní. Asi nejznámější příkladem vrstvové architektury je model síťové komunikace ISO/OSI. V obchodních aplikacích se běžně používá třívrstvý model (prezentace, doména, zdroj dat) [5].

Vrstvový model není populární jen v informatice, používá se také např. v teoretické lingvistice. Jedné z lingvistických teorií, které počítají s vrstvovým modelem – funkčnímu generativnímu popisu – je věnována část kapitoly 4.2.

V této práci jsme se rozhodli pro použití vrstvového modelu API. V tabulce 4.1 se nachází stručný přehled použitých vrstev. Jejich přesnější popis následuje v dalších podkapitolách.

Popis jednotlivých vrstev

Fyzická vrstva

Tato vrstva zajišťuje nízkoúrovňovou komunikaci s ovládaným zařízením. Konkrétní implementace (např. používaný protokol) je pro ostatní vrstvy skrytá. Komponenty, které jsou součástí jiných vrstev, můžou přistupovat k fyzické vrstvě pouze pomocí abstraktního rozhraní, které je závislé na konkrétním ovládaném zařízení, nikoli ale na implementačních detailech ovladačů. Toto rozhraní poskytuje metody pro ovládání zařízení a umožňuje reagovat na nastalé události.[1] V ideálním případě nemusí být programátorům, kteří pracují na ostatních vrstvách, o konkrétní implementaci ovládání robota nic známo.[2]

[1] Programovací jazyk Java, ve kterém jsme provedli konkrétní implementaci API, nemá přímou podporu pro odchytávání událostí. Místo nich používáme návrhový vzor *Observer* [14].

[2] V praxi je bohužel tento požadavek splněn málokdy. Programátor musí např. vědět, zda se pro ovládání zařízení používá síťový přenos dat (což je případ robota Nabaztag:tag), jestli by příkazy měly být seskupovány do větších balíků atp.

Vstup (uživatel → zařízení)	Komponenta	Výstup (zařízení → uživatel)
Přímé ovládání zařízení.	**Fyzická vrstva**	Vygenerování události.
Interpretace zpráv v mezijazyce a volání konkrétních metod fyzické vrstvy.	**Interpretační vrstva**	Reakce na události a vytváření parametrizovaných zpráv v mezijazyce na jejich základě.
Porozumění zprávě v přirozeném jazyce (rozpoznání mluvené řeči nebo parsování textu a vytvoření odpovídající zprávy v mezijazyce).	**Vrstva porozumění jazyku**	Vytvoření textové nebo zvukové zprávy v přirozeném jazyce na základě parametrizované zprávy v mezijazyce.
Příjem uživatelského vstupu (typicky se jedná o textové nebo zvukové zprávy).	**Uživatelské rozhraní**	Zobrazení nebo přehrání zprávy v přirozeném jazyce uživateli.

Tabulka 4.1: Přehled vrstvového modelu API pro ovládání robota v přirozeném jazyce.

Interpretační vrstva

Na této vrstvě dochází k *interpretaci* příkazů/zpráv v mezijazyce (viz 4.2), tj. k volání metod rozhraní poskytovaného fyzickou vrstvou na základě zpráv v mezijazyce. Interpretační vrstva dále odpovídá za vytváření zpráv v mezijazyce[3]

[3]Obecné generování zpráv na základě sémantické reprezentace je velmi komplikovaný problém. I kdyby se podařilo vyřešit ho pro některé jazyky, lokalizace softwaru do dalších jazykových prostředí by vyžadovala obrovské množství práce. Z těchto důvodů řešíme generování zpráv na této úrovni následujícím způsobem: k dispozici máme indexované slovníky vzorců pro

na základě událostí generovaných fyzickou vrstvou.

Interpretační vrstva je obecnější než fyzická vrstva, nejedná se ale o zcela komponentu nezávislou na ovládaném zařízení. Lze ji sdílet napříč zařízeními, které poskytují podobnou funkcionalitu a tím pádem zpřístupňují stejná (nebo podobná) rozhraní. Může se jednat např. o autorádia, GPS navigace nebo internetové prohlížeče.

Vrstva porozumění jazyku

V této vrstvě dochází ke konverzi mezi uživatelským vstupem v přirozeném jazyce (textem nebou mluvenou řečí) a univerzálním mezijazykem. Součástí této vrstvy by mělo být:

- porozumění psanému textu (parsování a interpretace) – konverze text → mezijazyk
- rozpoznání mluvené řeči – konverze zvuk → mezijazyk
- vygenerování zprávy v přirozeném jazyce – konverze mezijazyk → text nebo řeč

Zpráva se po konverzi do mezijazyka předává interpretační vrstvě. Teoreticky by vrstva porozumění jazyku mohla být nezávislá na ovládaném zařízení. Nicméně kvůli komplikovanosti zpracování zejména zvukových zpráv v přirozeném jazyce ji lze sdílet jen v rámci společné jazykové domény a měla by mít přístup ke kontextu.

generování textových zpráv (podrobnosti viz kapitola 6.8.6 na stránce 85). Místo aby zpráva byla generována v mezijazyce, komponenta vybírá index vzorce a určuje parametry. Pro lokalizaci takto vzniklých hlášek stačí přeložit lokalizační slovník do cílového jazyka.

Uživatelské rozhraní

Tato vrstva zajišťuje přímou komunikaci s uživatelem, především textový a/nebo zvukový vstup a výstup. Uživatelský vstup se dále předává interpretační vrstvě. Vrstva uživatelského rozhraní je obecná a nezávislá na ovládaném zařízení – odvíjí se pouze od způsobu zadávání příkazů uživatelem.[4]. Součástí této vrstvy může být i generování řeči z psaného textu. Volitelné dodatečné součásti této vrstvy zahrnují např.:

- konzola nebo grafický prvek uživatelského rozhraní, který umožňuje zadání textového vstupu a zobrazení textového výstupu
- komponenty pro pořizování zvukového záznamu pomocí mikrofonu počítače nebo ovládaného zařízení
- ovladače reproduktorů počítače nebo robota

4.2 Mezijazyk

Implementaci API, které umožňuje ovládání robota v přirozeném jazyce, významně zjednodušuje použití sémantického popisu, jenž vychází z koncepce tektogramatické roviny a funkčního generativního popisu [6]. Pro potřeby této práce specifikujeme *mezijazyk* – zjednodušenou obdobu tektogramatické roviny. Mezijazyk stojí na výstupu všech komponent API, které slouží porozumění textu a

[4]Je nicméně možné, že vstupní nebo výstupní zařízení bude součástí ovládaného robota (např. do výbavy robota Nabaztag:tag patří mikrofon, pomocí kterého lze zaznamenávat uživatelský vstup, a reproduktory, na které lze poslat výstup). V tomto případě musí vrstva uživatelského rozhraní volat metody z fyzické vrstvy.

rozpoznávání řeči (vrstva porozumění jazyku). V této kapitole popíšeme rozčlenění jazyka do rovin ve funkčním generativním popisu i návrh našeho mezijazyka.

Funkční generativní popis

Funkční generativní popis (FGP) je teoretický model jazyka založený na závislostním formalismu.

Základem FGP je rozčlenění popisu přirozeného jazyka do čtyř rovin [6].[5] Sousední roviny stojí ve vzájemném vztahu formy a funkce (nižší rovina je formou vyšší roviny, kdežto vyšší rovina je funkcí roviny nižší).

Tektogramatická rovina reprezentuje hloubkový význam sdělení a neobsahuje idiomy a řeší nejednoznačnosti. Je společná pro všechny přirozené jazyky.

V této rovině bude mít např. české sdělení *jedu do Kolína* různé reprezentace v závislosti na tom, zda se jedná o Kolín ve Středočeském kraji nebo Kolín nad Rýnem.

Rovina povrchové syntaxe zachycuje formální strukturu věty (podmět, přísudek, valenční rámec, volná doplnění). Na rozdíl od tektogramatické roviny je závislá na konkrétním jazyce a nerozlišuje význam slov (může tím pádem obsahovat víceznačnosti).

V rovině povrchové syntaxe má sdělení *jedu do Kolína* stejnou reprezentaci nezávisle na přesném cíli cesty.

[5] Starší verze FGP pracovaly dokonce s pěti rovinami (roviny fonetická a morfonologická byly sloučeny do společné fonologické roviny).

Morfematická rovina popisuje morfémy[6], ze kterých se skládá věta. Mezi morfémy patří:

- lexikální morfémy (kmeny slov),
- gramatické morfémy (pádové a slovesné koncovky nebo např. gramatické znaky v aglutinačních jazycích),
- odvozovací morfémy (předpony a přípony, které mění význam slova).

Fonologická rovina zachycuje sdělení jako posloupnost hlásek nebo posloupnost fonémů.

Hláska je základní řečová jednotka z hlediska fonetiky. Jedná se o množinu velmi podobných zvuků vyslovených člověkem. Např. *n* ve slově *ne* a *n* ve slově *banka* jsou dvě různé hlásky (`[n]` a `[ng]`).

Foném je jednotka vyšší úrovně než hláska, je to minimální zvuková jednotka rozlišující zvuk. Jedná se o množinu tzv. alofonů – různých hlásek, které neovlivňují význam. Např. `[n]` a `[ng]` nejsou v českém jazyce odlišné fonémy.

Implementace mezijazyka

Vyjádření v mezijazyce jsou implementována jako stromová struktura s jasně určeným kořenem. Všechna vyjádření mají určený kořen. Každý uzel mezijazyka je trojice (slovo, atributy, potomci), podrobnější vysvětlení se nachází v tabulce 4.2. Přesná struktura (sada používaných slov a atributů, způsob přiřazení potomků rodičům) je závislá na konkrétní implementační vrstvě. V tabulce 4.3 uvádíme příklady interpretace českých vět v ukázkové implementaci mezijazyka.

[6]Morfém je nejmenší jazykový jednotka nesoucí význam.

Prvek	Popis
lemma	Hlavní význam uzlu.
atributy	Asociativní seznam klíč–hodnota. Jednotlivé atributy můžou modifikovat význam *slova* (mj. určit, zda se jedná o příkaz nebo dotaz, přidat negaci atp.).
potomci	Asociativní seznam klíč–uzel. Jednotliví potomci představují parametry *lemmatu* (naplnění valenčního rámce, příslovečná určení atp.).

Tabulka 4.2: Popis uzlu vyjádření v mezijazyce.

4.3 Základní datové typy

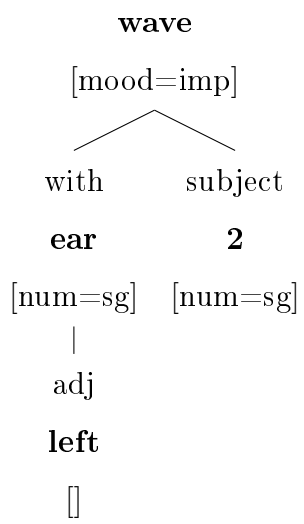
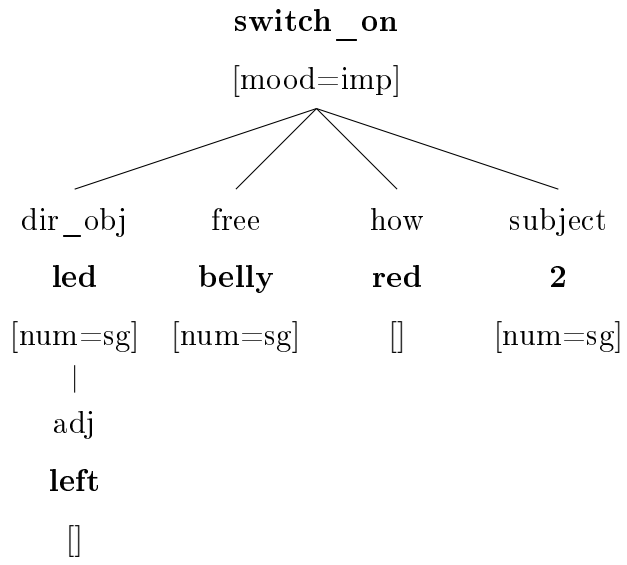
Pro potřeby API byly vytvořeny dva datové typy, které umožňují reprezentaci lingvistických objektů: textu v přirozeném jazyce a zvukového záznamu řeči. Základní třídou je `cz.cuni.mff.ufal.volk.Expression` (vyjádření v přirozeném jazyce), od které dědí třídy:

- `cz.cuni.mff.ufal.volk.Text` – texty v přirozeném jazyce,
- `cz.cuni.mff.ufal.volk.Speech` – záznam mluvené řeči.

Vlastnosti jednotlivých typů jsou popsány v tabulce 4.4 na stránce 44.

4.4 Služby

Pro řešení úloh, které nesouvisí přímo s ovládáním robota, nýbrž s obecnějším zpracováním přirozeného jazyka, jsme navrhli sadu lingvistických rozhraní

Věta	Interpretace
mávni levým uchem	
červeně rozsviť levé světýlko na bříšku	

Tabulka 4.3: Příklady interpretace českých vět v mezijazyce. Z příkladů byly pro zvýšení čitelnosti vynechány atributy, které nehrají pro interpretační vrstvu roli.

(*interfaců* jazyka Java). Tato rozhraní umožňují napojení jak vlastních implementací, tak komponent třetích stran (včetně vzdálených systémů), které řeší

`cz.cuni.mff.ufal.volk.Expression`

Vlastnost	Popis
`language`	Označení jazyka vyjádření podle normy ISO 639-1 nebo ISO 639-2 (přednost mají kratší kódy ISO 639-1). Tato vlastnost nemusí být nastavena (může být `null`), není-li jazyk vyjádření znám.

`cz.cuni.mff.ufal.volk.Text`

Vlastnost	Popis
`text`	Znakový řetězec (`String`) – text v přirozeném jazyce.

`cz.cuni.mff.ufal.volk.Speech`

Vlastnost	Popis
`format`	Název použitého formátu uložení zvuku podle `http://sox.sourceforge.net/soxformat.html`, např.: • *wav/signed-integer* – nekódovaný WAVE formát • *wav/oki-adpcm* – WAVE formát s ADPCM2 kódováním • *mp3*
`data`	Binární data (pole bajtů) – zvukový záznam v příslušném formátu. Přesná specifikace záleží na použitém formátu.

Tabulka 4.4: Základní popis vlastností datových typů pro reprezentaci *vyjádření* v přirozeném jazyce (text, záznam řeči). Vypsané vlastnosti slouží pouze pro čtení (v konkrétní implementaci v Javě přístupné pomocí metod `get`...).

zadaný problém. Všechna níže popsaná předdefinovaná rozhraní jsou součástí balíku `cz.cuni.mff.ufal.robot.services`.

Rozpoznávání jazyka

Pro komponenty, které umožňují rozpoznávání jazyka textu nebo zvukového záznamu (zatím pouze rozhraní, chybí konkrétní implementace), jsou určena generická rozhraní `LanguageRecognizer` a `LanguageRecognizerExtended`. V níže uvedených ukázkách kódu `T` je text v přirozeném jazyce nebo záznam řeči (obecně libovolný typ odvozený od `Expression`).

`String recognize(T item)` Metoda je součástí obou zmíněných rozhraní. Vrací kód (ISO 639-1[7] nebo ISO 639-2[8]) jazyka, který je nejpravděpodobnější pro vyjádření `item`.

`List<Hypothesis> recognize(T item, int max)` Metoda je zpřístupňována rozhraním `LanguageRecognizerExtended`. Vrací nejen kód nejpravděpodobnějšího jazyka, nýbrž až `max` nejlepších hypotéz (kód jazyka a hodnocení).

Součástí této práce je komponenta pro statistické rozpoznávání jazyka textu (rozpoznávání jazyka zvukového záznamu nebylo zatím implementováno) a konfigurační soubor (výsledek trénování), který umožňuje rozpoznávání angličtiny, češtiny, slovenštiny, polštiny a kašubštiny[9] (slovanské jazyky s diakritikou nebo

[7]Norma ISO 639-1 určuje 136 dvoupísmenných zkratek jazyků. Např. čeština je reprezentována kódem `cs` a angličtina – `en`.

[8]Norma ISO 639-2 určuje 464 třípísmenných kódů jazyků (určuje mj. kódy jazyků zohledněných v normě ISO 639-1.). Např. českému jazyku odpovídá v této normě kód `cze`, anglickému – `eng`, kašubštině – `csb` (kašubština nemá kód ISO 639-1).

[9]Kašubština je jazyk nebo nářečí polštiny (záleží na zdroji), kterým mluví cca. 50 000 lidí, především na severu Polska.

bez ní).

Generování řeči z psaného textu

Generování řeči z psaného textu je možné pomocí komponent, které implementují rozhraní `TextToSpeech`. Rozhraní zpřístupňuje následující metody:

`List<String> availableVoices(String language)` Vrací seznam hlasů podporovaných pro jazyk `language`.

`Speech process(Text text, String voice)` Generování řeči pomocí zadaného (parametr `voice`) hlasu (jazyk by měl určovat parametr `text`). Metoda vrátí zvukový záznam, který odpovídá textu obsaženému v parametru `text`. Formát výstupu záleží na použité komponentě.

V této práci pro generování řeči z textů v českém jazyce používáme program Epos (viz kapitola 5.7;pomocí obálky `EposTextToSpeech`). Zvukový výstup je ve formátu WAVE.

Sémantická analýza sdělení v přirozeném jazyce

Komponenty, které umožňují interpretaci sdělení v přirozeném jazyce, implementují generické rozhraní `Understander<T>`. Toto rozhraní je součásti vrstvy porozumění přirozenému jazyku a zpřístupňuje jednu metodu:

`Node understand(T item)` – tato metoda vrátí sémantickou analýzu sdělení `item` v mezijazyce.

Ukázková implementace existuje pouze pro textové záznamy v českém jazyce (jednoduchá analýza českých vět v Prologu, viz kapitola 6.8.3), rozpoznávání mluvené řeči nebylo zatím implementováno.

Konverze zvukových formátů

Komponenty, které poskytují možnost konverze mezi různými formáty zvukových záznamů, implementují rozhraní `SpeechConverter`. Nejedná se o funkci, která je přímo spojena se zpracováním přirozeného jazyka, nicméně často je konverze zvukového záznamu potřeba, aby vyhověl požadavkům používaných komponent pro rozpoznávání řeči.

`Speech convert(Speech input, String targetFormat)` Je to nejdůležitější metoda rozhraní, provádí konverzi zvukového záznamu do formátu určeného pomocí parametru `targetFormat` (je-li daný pár formátů podporován).

`List<String> sourceFormats()` Vrátí seznam všech zdrojových formátů.

`List<String> outputFormats()` Vrátí seznam všech cílových formátů.

`List<String> sourceFormats(String targetFormat)` Vrátí seznam těch zdrojových formátů, ze kterých je možná konverze do formátu `targetFormat` (podporovány můžou být konverze jen mezi některými páry zdrojových a cílových formátů).

`List<String> outputFormats(String sourceFormat)` Vrátí seznam cílových formátů, do kterých je možná konverze z formátu `sourceFormat`.

V této práci pro změnu formátu používáme program SoX[**?**] (pomocí obálky `SoxSpeechConverter`).

4.5 Ovladač králíka Nabaztag:tag

Ovladač umožňuje z programového kódu vyšších úrovní ovládání jednoho nebo více robotů současně. Nezávisí na externích poskytovatelích služeb (např. na serverech výrobce), vyžaduje pouze přímé síťové spojení mezi roboty a počítačem, kde běží ovladač. Zajišťuje obousměrnou komunikaci mezi robotickými králíky a počítačem. Může pomocí posluchačů (návrhový vzor *listener* [14]) reagovat na následující události:

- události, které králíci generují v pravidelných intervalech (pingy),
- zmáčknutí tlačítka uživatelem,
- nastavení pozic uší uživatelem,
- nahrání zvukového záznamu.

Pomocí ovladače je možné posílat králíkům příkazy a řídit tyto druhy chování:

- rozsvícení světýlek,
- nastavení pozic uší,
- přehrání zvukového záznamu,
- přehrávání internetového rádia nebo jiného streamovaného zvuku v podporovaném formátu,
- přehrání choreografie.

Ovladač funguje jako server, který s ovládanými zařízeními komunikuje pomocí bezstavového protokolu HTTP. Z toho vyplývá omezení týkající se odesílání

požadavků králíkům – příkaz je možné odeslat pouze v odpovědi na HTTP dotaz od robota.

Ovladač je thread-safe.[10] Musí se pouze dbát na to, aby k volání jednotlivých metod docházelo ve správném stavu (např. nastavení ovladače lze měnit pouze před zapnutím nebo po vypnutí (viz kapitola 6.8.5 na stránce 75).

[10] To znamená, že není potřeba synchronizovat vícevláknový přístup.

Kapitola 5

Použité technologie

Tato kapitola obsahuje přehled technologií, komponent a nástrojů třetích stran, které byly použity v této práci.

5.1 Java[1]

Pro implementaci API jsme pro jeho dostupnost na mnoha platformách zvolili programovací jazyk Java a platformu Java SE (verze 1.6 nebo 1.7).

5.2 Eclipse 3.5 a Eclipse 3.6[2]

Eclipse je integrované vývojové prostředí (IDE) šířené pod licencí Eclipse Public License, díky níž je volně dostupné i pro komerční účely. Je jedním z nejpopulárnějších IDE pro programovací jazyk Java, ale seznam podporovaných jazyků lze rozšířit díky mnoha dostupným zásuvným modulům.

[1] `http://java.com/en/`

[2] `http://www.eclipse.org`

5.3 Spring Framework 3.0.5[3]

Spring je framework pro vývoj Java aplikací, z něhož jsme v této práci použili Inversion of Control kontejner.

Inversion of Control (IoC) [8] je způsob programování, kdy se závislosti mezi objekty předávají pouze pomocí argumentů konstruktorů, argumentů továrních metod nebo *vlastností*. Objekty potom spravuje externí systém (IoC kontejner), který vytváří celou strukturu závislostí sám, typicky na základě deklarativního popisu v konfiguračním souboru. Tímto způsobem nevzniká monolitická aplikace, nýbrž sada stavebních bloků, ze kterých lze na základě konfiguračních souborů stavět hotová řešení.

Níže vysvětlujeme několik pojmů používaných v souvislosti s IoC ve Spring Frameworku:

Bean je jakýkoliv objekt, který je spolu se svými závislostmi vytvářen a spravován IoC kontejnerem.

Vlastnost (property) třídy je speciální členská proměnná třídy, ke které programátor nepřistupuje přímo, nýbrž při jejím čtení a nastavování dochází k volání metod, které můžou např. zvalidovat novou hodnotu, vygenerovat událost, která může být dál zpracovávána, v případě potřeby vygenerovat výjimku atp. Java, na rozdíl od některých jiných programovacích jazyků (např. C#, nemá podporu vlastností na úrovni jazyka, proto se používá dvojice přístupových metod *getter*/*setter*.[4] Např. vlastnost `xxx` bude v kódu vypadat takto:

[3] `http://www.springsource.org/`

[4] Jelikož vlastnosti, které jsou dostupné pouze pro čtení, nejsou přímo nastavitelné pomocí Spring Frameworku, nezajímají nás gettery bez odpovídajících setterů.

```
public T getXxx() { /* implementace */ }
public void setXxx(T value) { /* implementace */ }
```

Aplikační kontext je objekt, který se primárně stará o vytváření a ničení beanů. Typicky pro celou aplikaci existuje jenom jeden aplikační kontext (lze použít návrhový vzor Singleton[14]).

Nyní následují příklady definic konkrétních beanů se stručnými popisy (viz příklady 5.1 až 5.4). Všechny příklady jsou převzaty ze skutečného konfiguračního souboru aplikace `volkapp` (viz kapitola 6.8.2).

```
<bean id="volk.stdout"
   class="cz.cuni.mff.ufal.ConsoleUserTextOutput" />
```

Příklad 5.1: Vytvoření jednoduchého beanu voláním výchozího (neparametrického) konstruktoru bez nastavování vlastností. Bean bude v systému (v aplikačním kontextu a pro ostatní beany) dostupný pod indexem `volk.stdout`.

```
<bean id="volk.tts" destroy-method="close"
   class="cz.cuni.mff.ufal.volk.services.EposTextToSpeech">
  <constructor-arg
     value="C:\Program Files (x86)\Epos_2.4\bin\epos.exe"/>
  <constructor-arg value="4444"/>
</bean>
```

Příklad 5.2: V tom případě se pro vytvoření beanu použije konstruktor se dvěma parametry. Navíc se později během vypínání aplikace zavolá metoda `close` tohoto beanu. Bean je dostupný pod indexem `volk.tts`.

```
<bean id="feed.ihned"
      class="cz.cuni.mff.ufal.volk.RssInputProcessor.
         FeedConfiguration">
  <constructor-arg value="http://ihned.cz/?p=000000_rss" />
  <property name="encoding" value="windows-1250" />
</bean>
```

Příklad 5.3: Vytvoření beanu pomocí konstruktoru s jedním parametrem typu `String`. Navíc hned po vytvoření beanu dojde k nastavení vlastnosti `encoding`.

```
<util:map id="services">
  <entry key="volk.tts" value-ref="volk.tts" />
  <entry key="volk.langrec" value-ref="volk.langrec" />
  <entry key="volk.stdout" value-ref="volk.stdout" />
  <entry key="volk.sndconv" value-ref="volk.sndconv" />
</util:map>
```

Příklad 5.4: Vytvoření asociativního seznamu.

5.4 Log4j 1.2.16[5]

Logování je zápis informací o běhu programu. Nejčastěji jsou tyto informace ukládány jako textové záznamy do souborů nebo vypisovány na standardní výstup. Logování událostí má dva hlavní účely:

1. Umožnit uživatelům systému získat přehled o činnosti programu; zejména o chybách, ke kterým došlo za běhu programu.

[5] `http://logging.apache.org/log4j/1.2/`

2. Zjednodušit programátorům vývoj – umožnit např. odstraňování chyb, které se projevují jenom v produkčním prostředí, kde není dostupný debugger.

Všechny naprogramované komponenty používají pro logování knihovnu log4j. Existují uživatelské nástroje, které umožňují s logy generovanými touto knihovnou jednoduše pracovat – poskytují možnost je třídit a filtrovat podle různých kritérií.

Knihovna je široce konfigurovatelná a umožňuje měnit nastavení logování bez nutnosti upravovat zdrojový kód programu a vydávat různé verze podle požadavků, které se můžou velmi často měnit (zejména během vývoje a debugování pluginů). Příklad 5.5 na stránce 55 ukazuje zdrojový kód v Javě pro jednoduché volání loggeru log4j. Logování je možné konfigurovat globálně pro celou aplikaci nebo na úrovni jednotlivých balíků a tříd. Konfigurace knihovny je standardně uložena v XML nebo v souboru *properties* (viz příklad 5.6 na stránce 55). Konfigurovat lze mj.:

požadovanou úroveň logování Logují se všechny události dané nebo vyšší důležitosti (knihovna obsahuje definice následujících úrovní: TRACE, DEBUG, INFO, WARN, ERROR, FATAL, OFF; je možné si nadefinovat i další úrovně, nicméně se to nedoporučuje); různé třídy můžou mít nastaveny různé úrovně.

appendery Výstupy logování (např. standardní výstup, textový soubor, XML soubor, TCP/IP socket, systémový log, zápisy událostí systému MS Windows, SMTP; lze naprogramovat vlastní výstupy).

formát dat Umožňuje určit např. formát času, čísel atp.

```
// declaration:
private static final org.apache.log4j.Logger log =
  org.apache.log4j.Logger.getLogger(Main.class);
// usage:
log.info("Hello world!!!");
```

Příklad 5.5: Příklad definice a použití loggeru ve třídě `Main`.

```
# Logujeme všechno od úrovně TRACE.
log4j.rootLogger=TRACE, A1, C

# Logujeme do souboru volkapp.log...
log4j.appender.A1=org.apache.log4j.RollingFileAppender
log4j.appender.A1.File=volkapp.log
log4j.appender.A1.layout=org.apache.log4j.PatternLayout
# ... a na standardní výstup
log4j.appender.C=org.apache.log4j.ConsoleAppender
log4j.appender.C.layout=org.apache.log4j.PatternLayout

# Na standardní výstup logujeme pouze varování a chyby. Hlášky
# nižších úrovní se na standardní výstup nebudou logovat.
log4j.appender.C.threshold=WARN

# Časová značka ve formátu ISO 8601
log4j.appender.A1.layout.ConversionPattern=%d [%t] %-5p %c -
    %m%n
```

```
# Z balíčku org.springframework se budou zobrazovat pouze
# hlášky od úrovně WARN.
log4j.logger.org.springframework=WARN
```

Příklad 5.6: Ukázkový konfigurační soubor knihovny log4j. Pro přehlednost jsme se rozhodli používat formát properties, nikoli XML.

5.5 JUnit[6]

JUnit je framework pro vytváření unit testů kódu v programovacím jazyce Java, integrovaný do vývojového prostředí Eclipse.

5.6 Prolog

Prolog neměl podle původního návrhu být obecně použitelným programovacím jazykem, byl zamýšlen jako nástroj pro strojové zpracování textu v přirozeném jazyce (konkrétně ve francouzštině [2]). V dnešní době se Prolog používá nejen v počítačové lingvistice, nýbrž i v ostatních odvětvích umělé inteligence.

Existuje mnoho distribucí Prologu, mezi nejpopulárnější patří Sicstus[7] a SWI Prolog (šířený pod licencí GNU GPL). V této práci jsme použili dvě – tuProlog a SWI Prolog. Tabulka 5.1 na stránce 58 obsahuje stručné porovnání obou distrobucí Prologu se zohledněním jejich interoperability s programy běžícími ve virtuálním stroji Java.

[6] `http://www.junit.org/`

[7] `http://www.sicst.se`

tuProlog 2.4[8]

Tato implementace Prologu umožňuje spouštět programy a vyhodnocovat dotazy v tomto jazyce v rámci běžící instance virtuálního stroje Java. V naší práci je tuProlog použit v knihovně `nlpat`, která slouží generování textových zpráv na základě vzorců.

SWI Prolog 5.10.4[9]

SWI Prolog je jedním z nejpopulárnějších interpretů tohoto jazyka. Na rozdíl od předchozího vyžaduje instalaci nativního softwaru v operačním systému. Pro tuto práci jsme ho díky kvalitní podpoře DCG gramatik rozhodli použít v knihovně `nlu_prolog` (porozumění uživatelskému vstupu v přirozeném jazyce a jeho koverze do mezijazyka).

JPL[10]

Knihovna JPL zajišťuje spouštění programů napsaných v SWI Prologu a vyhodnocování dotazů v tomto jazyce z kódu Javy.

5.7 Epos 2.4.83[11]

Epos je volně dostupný systém pro generování řeči z psaného textu; vyžaduje instalaci do operačního systému. Výchozí instalace bez dodatečných prostředků

[8] `http://apice.unibo.it/xwiki/bin/view/Tuprolog/`
[9] `http://www.swi-prolog.org/`
[10] `http://www.swi-prolog.org/packages/jpl/`
[11] `http://epos.ure.cas.cz/`

	SWI Prolog s JPL	tuProlog
Interoperabilita	Vyžaduje instalaci do operačního systému.	Knihovna vytvořená v Javě bez použití nativních komponent.
Výkon	Vyhovující.	Významně klesá v případě větších bází znalostí
Spolehlivost	Vyhovující.	Vyžaduje zlepšení, chyby jsou dokonce v parseru.
Podpora více instancí Prologu v rámci jedné aplikace	Chybí. Podpora více bází znalostí se ale dá simulovat např. tak, že se jednotlivé soubory (programy) načtou do modulů.	Libovolný počet instancí v rámci jedné aplikace. Jednotlivé instance můžou pracovat s úplně jinou teorií, s jinou sadou knihoven atd.
Možnost rozšiřovat aktuální teorii.	Chybí, načítat lze pouze kompletní zdrojové soubory.	Za běhu je možné jednoduše načítat další soubory, doplňovat definice predikátů atp.
Podpora DCG gramatik.	Velmi dobrá.	Nevyhovující.

Tabulka 5.1: Porovnání hlavních rysů SWI Prologu s knihovnou JPL a tuPrologu. Při generování zpráv na základě vzorců se zejména hodila možnost mít více enginů Prologu najednou, při porozumění textu pokročilá podpora DCG gramatik; proto jsme se rozhodli použít dvě různé, navzájem nekompatibilní, distribuce.

poskytuje podporu češtiny a slovenštiny. Funguje jako server, klienti s ním komunikují pomocí protokolu TTSCP[15].

5.8 SoX 14.3.2[12]

Sound eXchange (SoX) je program, který umožňuje konverzi mezi různými zvukovými formáty. Jako jedna z mála volně dostupných aplikací podporuje formát ADPCM2, v němž zaznamenává zvuk Nabaztag:tag. Vyžaduje instalaci do operačního systému.

5.9 Rome 1.0[13]

Rome je javovská knihovna pro práci se zdroji informací RSS. Umožňuje vysokoúrovňový přístup k informacím bez nutnosti parsovat XML.

[12] `http://sox.sourceforge.net/`

[13] `http://java.net/projects/rome/`

Kapitola 6

Implementace

Ukázková implementace API v jazyce Java a příslušná dokumentace automaticky vygenerovaná pomocí nástroje Javadoc je součástí této práce. Konkrétní implementace pracuje s robotickým Wi-Fi králíkem Nabaztag:tag.

6.1 Hardwarové a systémové požadavky

Níže uvádíme seznam hardwarových a softwarových požadavků, které musí být pro spuštění základní verze serveru a zprovoznění komunikace mezi robotem a tímto serverem splněny. Dodatečné pluginy můžou tyto požadavky rozšiřovat.

- Robot Nabaztag:tag (server nebyl testován s jinými verzemi robota).
- Počítač s Wi-Fi kartou a nainstalovaným frameworkem Java (Java Runtime Environment 1.6 nebo 1.7).
- Je-li na počítači, na němž má běžet server, zapnutý firewall, virtuální stroj Java musí mít povolena příchozí spojení na příslušném portu.

- Wi-Fi síť, ke které se můžou připojit robot a počítač. Nabaztag:tag podporuje připojení k síti bez zabezpečení, příp. s šifrováním WEP nebo WPA (nikoli WPA2).
- Pro funkčnost některých komponent (porozumění textu, generování zpráv na základě vzorců) je potřeba mít nainstalovaný SWI-Prolog 5.10.4.

6.2 Implementace jednotlivých vrstev

Fyzická vrstva

Fyzická vrstva je realizována ovladačem `volk`, který zajišťuje obousměrnou komunikaci mezi ovládanými zařízeními pomocí protokolu HTTP.

Interpretační vrstva

V rámci interpretační vrstvy dochází ke konverzi sémantického popisu sdělení do volání příslušných metod fyzické vrstvy. K tomu jsou určené komponenty, které implementují javovský interface `Listener<Node>`, např. třída `NabaztagSimpleInterpreter`.

Vrstva porozumění jazyku

Generický interface `Understander<T extends Expression>` umožňuje standardizovaný přístup k různým objektům, které odpovídají za porozumění sdělením v přirozeném jazyce. V této práci pojem *porozumění* výrazu znamená konverzi objektu typu `Expression` (představujícího text nebo zvuk) do objektu `Node`, který reprezentuje sdělení na tektogramatické rovině.

Propojení interpretační vrstvy a vrstvy porozumění jazyku

V rámci této práce jsme implementovali několik jednoduchých komponent, které integrují funkcionalitu obou zmíněných vrstev. Mezi ně patří:

- `cz.cuni.mff.ufal.volk.RssInputProcessor` – jednoduchá čtečka RSS zdrojů, robot předčítá nové zprávy,
- `cz.cuni.mff.ufal.volk.ReaderInputProcessor` – předčítání uživatelského vstupu.

Uživatelské rozhraní

Vrstva uživatelského rozhraní zajišťuje přímou komunikaci systému s uživatelem. Sestává z následujících komponent:

- `cz.cuni.mff.ufal.volkapp.UserTextInput` – komponenta pro načítání uživatelského vstupu v přírozeném jazyce z konzoly,
- ovladač `volk` – umožňuje jednak přehrávání hlášek v přirozeném jazyce uživateli, jednak zaznamenávání uživatelského hlasového vstupu (automatické rozpoznávání řeči v těchto záznamech není zatím implementováno).

6.3 Logování a trasování

V této práci používáme pro logování výhradně knihovnu log4j (viz kapitola 5.4 na stránce 53 a příklady 5.5 a 5.6).

Konvence pro logování, jimiž jsme se při vývoji serveru a nástrojů řídili:[1]

[1] Stejné konvence by se měly dodržovat i při vývoji pluginů.

- Nepoužívají se žádné jiné způsoby logování kromě log4j (není zejména povoleno libovolné vypisování hlášek na standardní výstup).
- Používají se pouze úrovně logování standardní v knihovné log4j (TRACE, DEBUG, INFO, WARN, ERROR, FATAL, OFF).
- Všechny logovací záznamy jsou v anglickém jazyce (s výjimkou logování uživatelského vstupu). V budoucnu lze uvažovat o lokalizaci záznamů v logu.
- Každá třída má svůj vlastní static (společný pro všechny instance) logger. Příklad 5.5 na stránce 55 ukazuje vytvoření a použití loggeru.[2]
- Konfigurace logování se nastavuje pouze v konfiguračním souboru. Nelze ji měnit z kódu.
- Doporučujeme neměnit formát logování určený v příkladu.[3]

Konfigurace logování v aplikaci `volkapp` (viz kapitola 6.8.2) je možná na třech úrovních:

1. Přednastavená konfigurace ve třídě `BasicConfigurator` (logování záznamů všech úrovní na standardní výstup).
2. Výchozí konfigurační soubor `volkapp.log4j.txt` – pokud existuje, aplikace se pokusí s jeho použitím konfigurovat logování.

[2]Pro zjednodušení práce doporučujeme přidat definici loggeru do šablony třídy v používaném vývojovém prostředí.

[3]Zvolený formát logování je vhodný pro jednoduché vyhledávání, třídění a strojové zpracování (na rozdíl od některých lokalizovaných formátů).

3. Konfigurační soubor, jehož cesta je zadána v beanu `log4j` v konfiguračním souboru aplikace (`volkapp.config.xml`) – pokud existuje, aplikace se pokusí pomocí něj konfigurovat logování.

Podporována je pouze konfigurace pomocí souborů typu *properties*. Selže-li krok 1 (což by se nikdy nemělo stát), aplikace se vůbec nespustí. Selže-li krok 2 nebo 3, bude použita poslední správná konfigurace. Loguje se i při načítání konfiguračního souboru Spring Frameworku (`volkapp.config.xml`), takže několik prvních záznamů bude zalogováno pomocí základní konfigurace, resp. pomocí konfigurace, která se nachází v souboru `volkapp.log4j.txt`.

6.4 Internacionalizace

V dnešní době se při vývoji aplikací klade velký důraz na přizpůsobení softwarového produktu pro uživatele z různých zemí, mluvící různými jazyky. Podle [13] je vývoj softwaru s podporou více jazyků založen na dvou prvcích:

Globalizace a lokalizovatelnost (*World-Readiness*) je způsob vývoje programů tak, aby je bylo možné jednoduše lokalizovat bez nutnosti zásahů do zdrojového kódu. Uživatelský vstup a výstup by měly být naprogramovány nezávisle na konkrétním přirozeném jazyce.

Lokalizace (*Localization*) je především překlad softwaru, ale také přizpůsobení aplikace k požadavkům konkrétního trhu. Kromě překladu hlášek určených pro uživatele se může jednat o změnu velikosti některých grafických prvků (např. kvůli tomu, že text v daném jazyce je kratší nebo delší než v původním), změnu rozmístění grafických prvků (např. při překladu ap-

likace do jazyka, který je zapisován zprava doleva), korektní formátování čísel a času atd.

Veškerá komunikace s uživatelem je připravena pro lokalizaci – všechna hlášené pro uživatele jsou umístěny do *.properties* souborů.

Informace v logu a zprávy obsažené ve výjimkác nejsou primárně určeny pro konečné uživatele, nýbrž pro administrátory a vyvojáře. Z tohoto důvodu logování a systém výjimek nebyly lokalizovány: všechny zprávy jsou v angličtině.

6.5 Debugování

Pro vývoj a debugování těch částí aplikace, které byly napsány v Javě, jsme používali vývojová prostředí Eclipse 3.5 a Eclipse 3.7[4] a standardní nástroje pro lokální debugování, které jsou součástí prostředí. K vývoji dalších nástrojů a ladění jejich spolupráce s ostatními moduly je možné zapojit vzdálené debugování (*remote debugging*).

Pro debugování kódu napsaného v Prologu jsme používali:

1. SWI-Prolog a Emacs[5] k debugování formální gramatiky češtiny.

2. Jednoduché spouštěcí prostředí tuPrologu vytvořené pomocí technologie Java Web Start[6] pro debugování kódu, který odpovídá za generování zpráv na základě vzorců. To sice neposkytuje tak pokročilé možnosti jako SWI-

[4]Vývojové prostředí je dostupné z: `http://www.eclipse.org/`. Dobrý úvod do problematiky debugování v prostředí Eclipse: `http://www.ibm.com/developerworks/library/os-ecbug/`.

[5]`http://www.gnu.org/s/emacs/`

[6]`http://tuprolog.sourceforge.net/jws/tuprolog2.4.0.jnlp`

Prolog, nicméně kvůli nekompatibilitě obou dialektů jazyka nelze obě prostředí libovolně zaměňovat.[7]

Na ladění a hledání chyb v jednodušších vzorcích stačí přidání nových unit testů, které upozorní programátora na případné rozdíly mezi očekávaným a skutečným výsledkem. Ve složitějších případech lze v konfiguraci aplikace aktivovat logování úrovně `DEBUG`, potom se bude při volání metody `evaluate` logovat prologovský kód, který se použije pro konkretizaci proměnných. Ten ze dál ladit samostatně.

6.6 Automatické testy

Pro následující komponenty byly sestaveny automatické testy:

`nlpat` – sada klasických unit testů, pracujících v prostředí JUnit frameworku,

`tool_langr` – testy úspěšnosti rozpoznávání jazyka psaného textu, nejedná se o jednotkové testy, nýbrž o samostatnou aplikaci.

6.7 Vydání distribuce

Pro vydání distribuce (sady spustitelných *jar* souborů), stejně jako pro vygenerování dokumentace *javadoc*, jsme používali standardní nástroje pro export, které jsou součástí vývojového prostředí Eclipse.

1. Aplikaci `volkapp` jsme vyexportovali jako spustitelný *jar* soubor[8] s volbou, aby se všechny závislé knihovny kopírovaly do podsložky.[9] Došlo k vytvo-

[7] Kód funkční v tuPrologu by měl správně běžet pod SWI Prologem, nikoli ale naopak.

[8] Ang. *Runnable JAR file.*

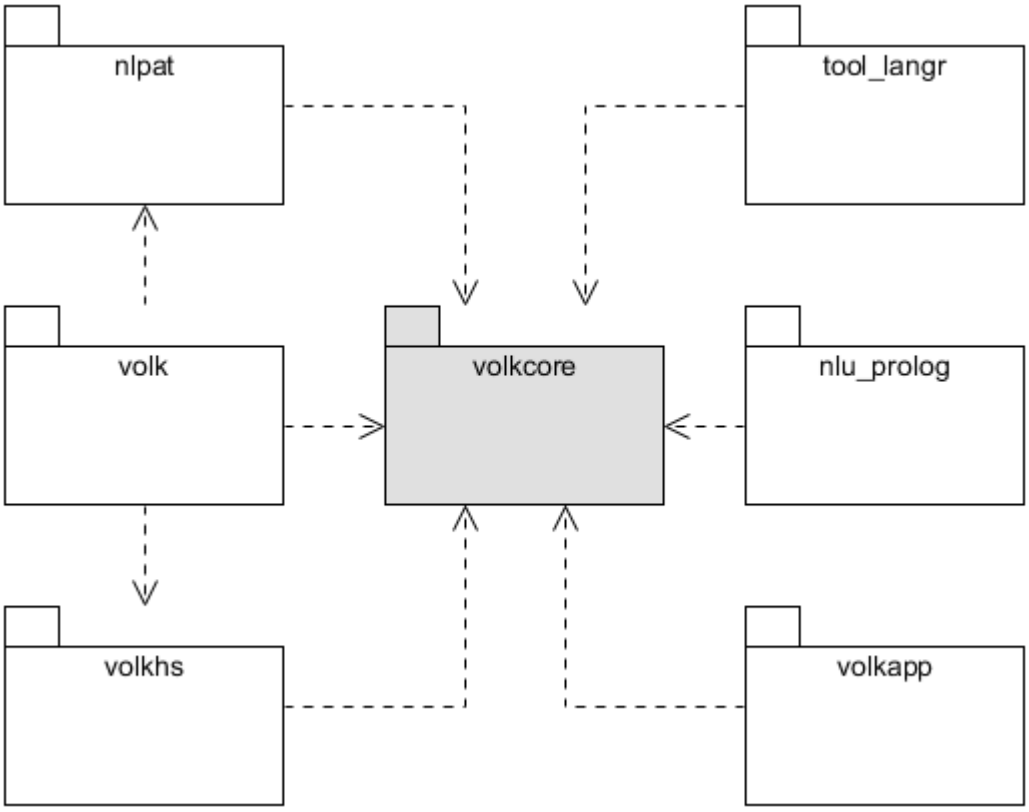
[9] Ang. *Copy required libraries into a sub-folder next to generated JAR.*

ření souboru `volkapp.jar` a složky `volkapp_lib`.

2. Ostatní knihovny jsme vyexportovali jako obyčejné *jar* soubory do složky `volkapp_lib` vytvořené v předchozím kroku.

Standardní dokumentaci *javadoc* jsme vyexportovali s výchozím nastavením najednou pro všechny knihovny.

6.8 Přehled komponent



Obrázek 6.1: Přehled implementovaných komponent a závislostí mezi nimi. Uprostřed se nachází knihovna `volkcore`, na které jsou závislé všechny ostatní komponenty.

6.8.1 Obecné API – `volkcore`

Tato knihovna je vyžadována všemi ostatními komponentami a obsahuje základní rozhraní (*interfaces* v Javě) a následující typy (viz obrázek 6.1):

- základní datové typy,
- rozhraní jednotlivých služeb,
- některé ukázkové implementace služeb,
- výjimky používané ostatními komponentami,
- pomocné typy, které nesouvisí přímo se zpracováním přirozeného jazyka.

Základní datové typy

Pro potřeby API pro ovládání robota byly vytvořeny datové typy, které reprezentují sdělení v přirozeném jazyce:

`cz.cuni.mff.ufal.volk.Expression` – abstraktní třída; sdělení v přirozeném jazyce,

`cz.cuni.mff.ufal.volk.Text` – text v přirozeném jazyce,

`cz.cuni.mff.ufal.volk.Speech` – záznam mluvené řeči.

Jejich návrh je popsán v kapitole 4.3 na stránce 42).

Mezijazyk

Součástí této komponenty je třída `cz.cuni.mff.ufal.volk.il.Node`, která implementuje sémantickou analýzu sdělení v mezijazyce. Návrh mezijazyka je popsán v kapitole 4.2.

Služby

Komponenta `volkcore` obsahuje obecná rozhraní služeb (*interfaces* v programovacím jazyce Java) i některé konkrétní ukázkové implementace:

`LanguageRecognizer, LanguageRecognizerExtended` – rozpoznávání jazyka sdělení v přirozeném jazyce,

`TextToSpeech` – automatické generování řeči z psaného textu,

`EposTextToSpeech` – generování řeči pomocí programu Epos (viz kapitola 5.7),

`SpeechConverter` – konverze zvukových formátů,

`SoxSpeechConverter` – implementace rozhraní `SpeechConverter`, konverze zvukových formátů pomocí programu SoX (viz kapitola 5.8),

`Understander` – sémantická analýza sdělení v přirozeném jazyce.

Všechny zmíněné typy jsou součástí balíčku `cz.cuni.mff.ufal.robot.services`. Návrh všech rozhraní je podrobně popsán v kapitole 4.4 na stránce 42.

Výjimky

Komponenta `volkcore` obsahuje následující typy výjimek:

`InitializationException` – chyba inicializace komponenty nebo aplikace (např. špatný konfigurační soubor),

`ProcessingException` – obecná chyba zpracování záznamu v přirozeném jazyce,

`UnsupportedInputTypeException` – tato výjimka informuje, že došlo k pokusu o zpracování záznamu ve formátu, který není podporován komponentou

(např. při pokusu o analýzu mluvené řeči komponentou, která podporuje pouze text),

`UnsupportedLanguageException` – k této výjimce dochází při pokusu o zpracování záznamu v přirozeném jazyce komponentou, která nepodporuje jazyk sdělení.

6.8.2 Aplikační framework – `volkapp`

Součástí této práce je jednoduchý aplikační kontejner `volkapp`, který spouští ovladač králíka Nabaztag:tag (viz 6.8.5) a poskytuje jednotlivé služby na základě konfigurace (kompatibilní se Spring Frameworkem). Konfigurace musí být v XML formátu a musí se nacházet v souboru `volkapp.config.xml` – ve složce, kde se nachází aplikace. Inicializaci, spuštění serveru, konfiguraci logování a zpřístupnění služeb zajišťuje aplikace, není tedy potřeba přistupovat přímo k objektům Spring Frameworku. Následuje seznam standardních konfiguračních položek – beanů (názvy povinných položek jsou uvedeny hvězdičkami; za dvojtečkou následuje typ beanu):

`*applicationSettings: ApplicationSettings` Globální nastavení aplikace. Zatím umožňuje určit pouze lokalizaci komunikace s uživatelem, především používaný jazyk.

`*components: List<Component>` Seznam komponent (objektů, které implementují rozhraní `cz.cuni.mff.ufal.volk.Component`). Ve výchozí konfiguraci obsahuje ovladač aplikace a komponentu, která umožňuje čtení a další zpracovávání uživatelského vstupu.

`log4j: String` Cesta k *properties* souboru, který obsahuje konfiguraci logování pro log4j framework (viz kapitola 5.4 a příklad 5.6). Pokud tato položka chybí, použije se výchozí konfigurační soubor `volkapp.log4j.txt`.

`*services: Map<String, Object>` Seznam zaregistrovaných služeb (jako mapování názvů na implementace). Tabulka 6.1 obsahuje výpis standardních služeb zaregistrovaných v systému ve výchozí konfiguraci.

Název	Popis
`volk.tts`	Generování řeči na základě psaného textu.
`volk.langrec`	Rozpoznávání jazyka textu nebo mluvené řeči (obecně objektu typu `Expression`).
`volk.sndconv`	Konverze mezi různými zvukovými formáty.
`volk.stdout`	Uživatelský standardní výstup (typicky konzola nebo prvek okna ovládací aplikace).

Tabulka 6.1: Standardní služby dostupné v systému.

Spuštění aplikace (volání static metody `cz.cuni.mff.ufal.volkapp.Main.main`) probíhá následujícím způsobem:

1. Načtení logování pomocí konfiguračních souborů (viz 6.3).
2. Vytvoření všech beanů ze seznamu `components` a jejich spuštění pomocí asynchronní metody `open()` z rozhraní `cz.cuni.mff.ufal.volk.Component`.

Aplikace běží, dokud některá komponenta nezavolá metodu `close()` objektu `cz.cuni.mff.ufal.volkapp.Application`. Při vypínání aplikace bude na každém spuštěném beanu zavolána metoda zaregistrovaná v konfiguračním souboru jako

destroy-method (existuje-li). Tyto metody by nikdy neměly generovat výjimky (všechny výjimky musí být odchyceny, příp. i zalogovány). Pořadí volání těchto metod není specifikováno.[10]

K inicializaci seznamu zaregistrovaných služeb (bean `services`) a globálních nastavení aplikace (bean `applicationSettings`) může dojít v libovolném okamžiku po spuštění aplikace, nejpozději však při prvním pokusu o jejich použití.[11] Implementace všech těchto metod umožňuje jejich bezpečné použití z více vláken bez nutnosti dodatečné synchronizace volání.

6.8.3 Rozhraní pro formální gramatiky – `nlu_prolog`

Tato knihovna umožňuje používání formálních gramatik napsaných v jazyce prolog pro porozumění uživatelskému vstupu. Vyžaduje SWI Prolog nainstalovaný v systému; teoreticky by knihovna `nlu_prolog` měla fungovat i s jinými distribucemi tohoto jazyka kompatibilními s JPL, byla ale testována pouze s SWI Prologem.

Parser přirozeného jazyka musí být naprogramován v Prologu a musí definovat proceduru `parse/2`, jejíž parametry jsou popsány v tabulce 6.2.

Součástí této práce je ukázková implementace – jednoduchý parser českého jazyka, který zpracovává mj. příkazy pro robotického králíka Nabaztag:tag– ukázka

[10] Tím pádem implementace těchto metod nesmí nic předpokládat o tom, zda už proběhlo volání pro jiné beany.

[11] Jedná se o volání jedné z následujících metod:

- `ApplicationSettings.getCurrent()`,
- `Services.registered()`,
- `Servives.get(name,T)`.



Obrázek 6.2: Adresářová struktura. Přehled obsahuje hlavní binární soubor `volkapp.jar`, povinné konfigurační soubory a závislosti.

Parametr	Popis
`+Sentence`	Seznam tokenů, který reprezentuje vstupní větu.
`-IL`	Výsledek parsování. Parametr se má unifikovat se strukturou `node(Sem, Attr, Children)`, kde: • `Sem` je tzv. význam uzlu – atom. • `Attr` je seznam atributů ve tvaru `Key-Value` (např. u slo-ves `mood-imp` znamná rozkazovací způsob). Pořadí atributů není důležité. • `Children` je seznam struktur `node/3`. Pořadí v seznamu `Children` může, ale nemusí hrát roli při interpretaci (zá-leží na konkrétní implementaci sémantické analýzyzž).

Tabulka 6.2: Specifikace parametrů procedury `parse/2`.

viz tabulka 6.3.

6.8.4 HTTP server – `volkhs`

`Volkhs` je jednoduchý, obecně použitelný HTTP server, který umožňuje nízkoúrovňové zpracovávání dotazů a generování odpovědí. Na požadavky může zpracovávat požadavky ve více vláknech (jejich maximální počet je konfigurovatelný).

Nejdůležitější třídy komponenty `volkhs`:

`HttpServer` Hlavní třída HTTP serveru. Zajišťuje nízkoúrovňovou komunikaci – příjem požadavků, jejich zpracování a odesílání odpovědí. Jedná se o imple-

Věta v češtině	Výsledek parsování
mávni levým uchem	Příslušná interpretace v mezijazyce (viz kapitola 4.2).
rozsviť levé světýlko červeně	
mávni levé uchem	Neúspěch kvůli chybnému pádu přídavného jména. Implementovaný parser češtiny neumí zacházet s nekorektně sestavenými větami.

Tabulka 6.3: Příklady výstupů ukázkového parseru českého jazyka. Na interpukci a velikosti písmen nezáleží. Česká diakritika je nicméně vyžadována.

mentaci návrhového vzoru pozorovatel [14]. Každá instance serveru musí mít právě jednoho pozorovatele typu `HttpRequestHandler`. Pozorovatele nelze pro vytvořenou instanci serveru změnit.

`HttpRequestHandler` Pozorovatel, který zpracovává konkrétní požadavky a generuje odpovědi. Poskytuje jednu metodu `process`, kterou server volá při přijetí HTTP požadavku. Její parametry jsou popsány v tabulce 6.4

6.8.5 Ovladač králíka Nabaztag:tag – `volk`

Ovladač zařízení je implementován jako třída `NabaztagControllerImpl` (součást knihovny `volk.jar`, balík `cz.cuni.mff.ufal.volk`), která může být použita libovolným programem (nebo knihovnou) napsaným v Javě. Umožňuje vysokoúrovňovou komunikaci s jedním nebo více roboty: příjem a odesílání zvuku, reakce na zmáčknutí tlačítka, rozsvěcení světýlek a nastavení pozic uší. Obousměrná přímá komunikace se samotným zařízením probíhá pomocí protokolu HTTP.

Parametr	Popis
`request`	Výsledek parsování HTTP dotazu – informace o použité metodě a hodnotách jednotlivých POST a GET parametrů.
`response`	Objekt, který umožňuje jak nastavit parametry odpovědi, mj. kód (typicky `200 OK`) a HTTP hlavičky, tak odeslat v jejím těle binární data.

Tabulka 6.4: Popis parametrů metody `HttpRequestHandler.process`.

Objekt ovladače může být sdílen více vlákny, není třeba synchronizovat přístup; ovšem třeba dbát na to, aby k volání jednotlivých metod docházelo pouze ve správném stavu – viz následující oddíl.

Stavy ovladače

Ovladač se může nacházet v jednom ze dvou stavů:

vypnuto V tomto stavu se ovladač nachází bezprostředně po svém vytvoření, nebo po zastavení pomocí metody `close()`. Ve stavu *vypnuto* ovladač nekomunikuje se zařízeními, umožňuje ale změnu nastavení.

zapnuto V tomto stavu ovladač poslouchá na definovaném portu a umožňuje komunikovat s roboty. Pokus o změnu nastavení v tomto stavu způsobí výjimku.

Přechody mezi stavy zajišťují metody `open()` a `close()`:[12]

[12]Pokus o zavolání těchto metod ve špatném stavu skončí výjimkou.

`void open()` Zapne server (ovladač), začne poslouchat na definovaném portu. Tuto metodu lze zavolat pouze ve stavu *vypnuto*. Skončí-li bez výjimky, ovladač se nachází ve stavu *zapnuto*.

`void close()` Vypne server. Lze ji zavolat pouze ve stavu *zapnuto*. Skončí-li bez výjimky, ovladač se nachází ve stavu *vypnuto*. Vypnutý ovladač lze znovu spustit pomocí metody `open`.

Aplikace by měla před ukončením běhu vypnout všechny používané ovladače (pro každý zavolat metodu `close()`). Nebude-li tato podmínka splněna, může dojít k zablokování portů a ovladače nebude možné do restartu počítače znovu spustit.

Ovládání robota

Níže uvádíme metody třídy `NabaztagProxy`, jejichž pomocí je možné ovládat robotické králíky Nabaztag:tag. Všechny metody lze zavolat, pouze když je server ve stavu *zapnuto*, jinak volání skončí výjimkou.

Všechny níže popsané metody jsou asynchronní – úspěšné zavolání metody znamená, že konkrétní požadavek byl zařazen do fronty a bude odeslán králíkovi v odpovědi na nejbližší *ping* požadavek (viz popis komunikace mezi serverem a králíkem na stránce 28). Níže uvedené metody odpovídají jednotlivým příkazům králíka:

`void reboot()` Restart králíka – následuje vypnutí a zapnutí robota, který se znovu přihlásí k serveru a znovu načte bootcode.

`void playSpeech(Speech s)` Požádá robota o přehrání záznamu zvuku `s`. Podporované formáty jsou: WAV (bez komprese nebo s kompresí ADPCM2) a

MP3.

`void playUrl(String url)` Pošle požadavek na přehrávání streamovaného zvuku (typicky internetového rádia). Podporovány jsou stejné formáty jako v případě `playSpeech`.

`void sendChoreography(String url)` Pošle požadavek na přehrání *choreografie* (jedná se o posloupnost příkazů, mezi kterými může být rozsvícení nebo vypnutí světla, změna pozic uší a přehrávání jednoduchých zvuků).

`void switchLedOn(Led led, Color c)` Pošle požadavek na rozsvěcení zvoleného světýlka na králíkovi.

`void switchLedOff(Led led)` Zařadí do fronty požadavek na vypnutí zvoleného světla na králíkovi.

`void moveEar(Ear e, int pos)` Požádá králíka o změnu pozice ucha. Parametr `pos` určuje novou pozici ucha (číslo v rozsahu 1 – 12).

`void setPingFrequency(int newFrequency)` Změna časového intervalu, v jakém králík kontroluje, zda na něj na serveru čekají nějaké požadavky. Výchozí hodnota je 10 vteřin. Čím menší je tato hodnota, tím rychleji bude králík reagovat na uživatelské příkazy.

Nastavení ovladače

Níže popisujeme možnosti nastavení ovladače. Všechna jsou přístupná pomocí getter a setter metod v následujícím tvaru (příklad pro `listenerAddress` typu `String`):

`String getListenerAddress()` Načte a vrátí hodnotu parametru.

`void setListenerAddress(String value)` Buď nastaví novou hodnotu parametru, nebo skončí výjimkou, je-li hodnota `value` nekorektní nebo k volání došlo ve špatném stavu.

Tento způsob přístupu ke konfiguraci umožňuje ovladač používat jako součást aplikace konfigurovanou prostřednictvím Spring Frameworku, viz 5.3. Jedná se o způsob kompatibilní s IoC kontejnerem, který zpřístupňuje Spring Framework a tím pádem je možné používat ovladač jako součást aplikace konfigurované pomocí tohoto frameworku.

Všechny volby musí být určeny před spuštěním ovladače (tj. před zavoláním metody `open()`), pozdější pokus o jejich změnu skončí výjimkou (server ale bude fungovat dál s původním nastavením). Nastavování parametrů serveru není thread-safe, v případě přístupu z více vláken je nutná synchronizace. Gettery lze volat v libovolném stavu (jejich volání by nikdy nemělo skončit výjimkou). Ovladač umožňuje nastavovat následující parametry:

`bootCodeLoader` Objekt, který umožňuje načtení bootcodu (operačního systému zařízení). Pro běžné použití by mělo stačit výchozí nastavení (bootcode bude vždy načten z resourců – `/res/bootcode.bin`). Server nebude fungovat s nejnovější verzí oficiálního bootcodu králíka Nabaztag:tag, která podporuje pouze protokol XMPP, nikoli HTTP komunikaci).

`listenerAddress: String` Adresa a port (není-li uveden port, předpokládá se standardní HTTP port 80[13]), na kterou Nabaztag:tag posílá HTTP požadavky.

[13]Typicky pro spuštění ovladače na portu 80 uživatel musí mít administrátorská práva.

`maxQueueSize: int` Maximální velikost fronty požadavků ke zpracování. Je-li tato hodnota ≤ 0, velikost fronty není omezena.[14] Výchozí hodnota je 10.

`maxQueueSizeExceededBehavior: Enum` Určuje chování ovladače v případě přijetí požadavku, jehož zařazení do fronty by způsobilo překročení její maximální velikosti. Možné hodnoty:

`RemoveOldRequests` Nový požadavek je zařazen na konec fronty, ze které je odstraněn nejstarší nezpracovaný požadavek (výchozí hodnota).

`IgnoreNewRequests` Fronta se nemění, nový požadavek do ní není zařazen.

`suppressMultipleEvents: boolean` Tato volba určuje způsob, jakým ovladač generuje události `soundRecorded`. Nabaztag:tag posílá každý nahraný zvuk několikrát (dvakrát až čtyřikrát). Událost `soundRecorded` může nastat pokaždé, když králík pošle záznam (nastavení `false`, výchozí), nebo pouze v případě, že daný záznam byl králíkem poslán poprvé (hodnota `true`).[15]

`threadPoolSize: int` Velikost fondu vláken (*thread pool*), zpracovávajících HTTP požadavky králíka. Tato hodnota musí být ≥ 1. Výchozí nastavení je 5.[16]

`workingFolder: String` Cesta k adresáři, kde bude ovladač vytvářet dočasné soubory (*pracovní adresář*). Je-li tento parametr `null` (výchozí), vytvoří se podsložka ve standardním temp adresáři; ta bude automaticky odstraněna

[14] Nicméně nedoporučujeme vypínat omezení velikosti fronty.

[15] Při tomto nastavení ovladač srovnává kontrolní součet nově přijatého zvuku s kontrolním součtem předchozího zvuku. Jsou-li tyto hodnoty odlišné, dojde k vygenerování události.

[16] Nedoporučujeme používat příliš malou hodnotu (1 nebo 2), může potom docházet ke zbytečně dlouhému čekání požadavků ve frontě.

při vypínání aplikace.[17]

`removeTempFiles: boolean` Určuje, zda dočasné soubory (např. vygenerované choreografie, výstup řečové syntézy) mají být po odeslání robotovi odstraněny. Výchozí hodnota je `true`.[18]

Události

Protože Java, na rozdíl od např. C#, neposkytuje přímou podporu událostí, pro rozesílání notifikací jiným komponentám používáme návrhový vzor *Pozorovatel – Observer* [14]. Příjemci musí implementovat rozhraní `NabaztagEventListener` (součást balíčku `cz.cuni.mff.ufal.volk`). Následující dvě metody umožňují změny seznamu notifikovaných komponent:

`void addListener(NabaztagEventListener o)` Pokud objekt `o` ještě nepatří do seznamu notifikovaných objektů, bude do něj přidán.

`void removeListener(NabaztagEventListener o)` Pokud je objekt `o` součástí seznamu notifikovaných objektů, bude z něj odebrán.

Obě výše zmíněné metody jsou synchronní (změny se projeví okamžitě po jejich zavolání) a můžou být volány v libovolném stavu ovladače.

Existuje rovněž možnost přidat kolekci posluchačů jako parametr konstruktoru:[19]

[17] Pevné nastavení cesty pracovního adresáře je vhodné např. při debugování ovladače nebo pluginů.

[18] Změna tohoto nastavení může být užitečná při debugování.

[19] Tento konstruktor jsme zpřístupnili kvůli kompatibilitě s IoC kontejnerem Spring Frameworku, díky tomu je možné určit kolekci posluchačů přímo v konfiguračním souboru aplikace.

`NabaztagControllerImpl(Collection<NabaztagEventListener> listeners)`

Rozhraní `cz.cuni.mff.ufal.volk.NabaztagEventListener` obsahuje následující metody:

`void onStarted(NabaztagProxy sender)` K volání této metody dochází po úspěšném odeslání bootcode (viz 3.3), možná i několikrát pro stejné zařízení (např. po restartu).

`void onClick(NabaztagProxy sender)` Metoda je volána po stisknutí tlačítka na hlavě králíka (pouze v případě, že se nejedná o nahrávání zvuku).

`void onDoubleClick(NabaztagProxy sender)` Metoda je volána poté, co uživatel stiskne dvakrát tlačítko na hlavě králíka.

`void onSoundRecorded(NabaztagProxy sender, Speech s)` Tato metoda je volána poté, co uživatel se zmáčknutým tlačítkem na hlavě králíka zaznamená zvuk. Nahraný zvuk je ve formátu WAVE s kódováním ADPCM2. K vygenerování této události může dojít pro jeden záznam několikrát (Nabaztag:tag posílá záznam tři- až čtyřikrát, důvod zatím neznáme). Lze tomu zabránit nastavením volby `suppressMultipleEvents` na `true`.

`void onCompleted(NabaztagProxy sender)` Tuto metodu server zavolá po úspěšném provedení konkrétního příkazu.

`void onTimeout(NabaztagProxy sender)` K volání této metody dojde poté, co Nabaztag:tag obdrží odpověď *timeout* ze serveru.

`void onEarsMoved(NabaztagProxy sender, int left, int right)` K volání metody dochází poté, co uživatel ručně změní pozici uší. Parametry `left`

a `right` určují novou pozici levého, resp. pravého ucha (hodnota 1 – 12, viz kapitola 3.3).

`void onVoiceInterrupted(NabaztagProxy sender)` Tato událost je generována poté, co uživatel přerušil přehrávání zvuku stiskem tlačítka na hlavě králíka.

Ovladač robota Nabaztag:tag je součástí knihovny `volk.jar`. Lze ho použít buď jako nezávislou aplikaci nebo jako objekt z kódu jiného programu. V obou případech ovládač funguje jako HTTP server postavený na `volkhs`. Je proto nezbytně nutné mít pro JVM povolena příchozí připojení na daném portu, jinak se Nabaztag:tag nedokáže připojit k serveru.

Použití ovladače

Pro použití ovladače z kódu jiného programu je třeba vytvořit objekt typu `cz.cuni.mff.ufal.volk.NabaztagController`. Programátor má dvě základní možnosti:

1. Samostatně naprogramovat vytváření objektu ovladače a předání všech potřebných závislostí. Tento postup je vhodný např. pro práci s konfigurací ve formátu nekompatibilním se Spring Frameworkem (viz příklad 6.1).

2. Použít standardní konfigurační soubor Spring Framework, což je možné díky kompatibilitě s principem IoC (viz příklad 6.2).

```
// povinné importy
import cz.cuni.mff.ufal.volk.NabaztagController;
//...
NabaztagControllerParameters params =
```

```
  new NabaztagControllerParameters();
params.setLocalDirectory("");
NabaztagController controller =
  new NabaztagControllerImpl(params);
controller.start();
// ...
controller.close();
```

Příklad 6.1: Ruční vytvoření a spuštění ovládače z kódu.

```
// povinné importy
import org.springframework.beans.factory.BeanFactory;
import org.springframework.beans.factory.xml.
  XmlBeanFactory;
import org.springframework.core.io.FileSystemResurce;
import cz.cuni.mff.ufal.volk.NabaztagController;
// ...
// předpokládáme, že proměnná inputFile je String a obsahuje
// název konfiguračního souboru
BeanFactory factory = new XmlBeanFactory(
  new FileSystemResource(inputFile));
NabaztagController controller = factory.
  .getBean("server", NabaztagController.class);
controller.start();
// ...
controller.close();
```

Příklad 6.2: Vytvoření a spuštění ovládače pomocí Spring Frameworku.

6.8.6 Generování sdělení na základě vzorců – `nlpat`

Knihovna `nlpat` umožňuje generovat zprávy na základě textových vzorců s použitím formálního popisu jazyka a slovníku. Její součástí je jednoduchý popis českého jazyka.

Vyhodnocování vzorců je implementováno v programovacím jazyce Prolog. Použili jsme prostředí tuProlog (viz 5.6).

Každý vzorec obsahuje *formátovací řetězec* a *podmínky*. K oddělování těchto částí se používá speciální znak `#`.

Formátovací řetězec

Formátovací řetězec je text s vnořenými parametry, které jsou po vyhodnocení nahrazovány konkrétními textovými řetězci. Parametry jsou ve formátovacím řetězci uvedeny v následujícím tvaru: `=název_parametru=`. Název parametru musí začínat malým písmenem, potom následuje posloupnost libovolně dlouhá posloupnost malých a velkých písmen anglické abecedy (a–z), číslic a podtržítek. Stejný parametr se může vyskytovat ve formátovacím řetězci vícekrát; i v takovém případě budě pokaždé nahrazen stejným textem.

Formátovací řetězec může obsahovat speciální znaky `#` a `=`. Tyto znaky musí ale být escapovány pomocí zpětného lomítka – \(samotné zpětné lomítko je též třeba escapovat – \\).

Podmínky

Podmínky jsou ve své podstatě definicí procedury `cond/1`[20] v jazyce Prolog. Pro jejich základní použití není nutná pokročilá znalost Prologu, stačí následující zjednodušené informace:[21]

Atomy (textové řetězce) buď začínají malým písmenem a obsahují pouze malá a velká písmena anglické abecedy, číslice a podtržítka, nebo jsou uzavřeny do uvozovek. Např. `case`, `nom`, `gen`, `'krásná'`, `'Praha'`.

Proměnné vždy začínají velkým písmenem a skládají se z malých a velkých písmen anglické abecedy, číslic a podtržítek. Např. `Name`, `Gen`.

Seznam je výčet prvků v hranatých závorkách. Jednotlivé prvky jsou odděleny čárkou. Např. `[a, b, c]` (seznam, který obsahuje tři atomy), `[X]` (libovolný jednoprvkový seznam), `[]` (prázdný seznam).

Klauzule můžou určovat buď *fakta* nebo *pravidla*:

Nápis	Význam
`s(a, b).`	Vždy platí `s(a, b).`
`s(A, B) :- c(A), d(A, B).`	Levá strana je pravdivá za předpokladu, že platí pravá strana.

Procedura je posloupnost klauzulí (alternativ) se stejnou aritou.

Řádkové komentáře začínají znakem `%`.

[20] Číslo za lomítkem označuje *aritu* (počet argumentů) objektu. V tomto konkrétním případě procedura `cond` vyžaduje jeden parametr.

[21] Stručný úvod do programování v Prologu v této kapitole je založen na [10].

Jediný argument procedury `cond/1` se unifikuje[22] se seznamem podmínek, které musí splňovat jednotlivé parametry ve formátovacím řetězci. Existují dva typy podmínek:

`Param -> SeznamVlastnosti` – jedná se o určení vlastností, které má splňovat parametr *Param* načítaný ze slovníku.

Všechny vlastnosti jsou ve tvaru `NazevVlastnosti == Hodnota`. Název vlastnosti musí být atom, hodnota může být buď atom nebo proměnná (hodí se např. pro určování shody).

`Param := Lit` – určení znakového řetězce, kterým má být nahrazen parametr *Param*. *Param* musí být konkrétní název parametru, nikoli proměnná.

Kvůli chybě parseru tuPrologu je třeba v podmínkách za tečku na konci klauzule vždy vkládat mezeru nebo znak nového řádku. Tuto skutečnost je třeba si pamatovat zejména při psaní víceřádkových vzorců přímo v kódu. Příklad 6.3 obsahuje definici procedury `cond/1` pomocí dvou klauzulí, které jsou odděleny pouze tečkou bez dodatečných bílých znaků. Parsování neproběhne korektně. Za první tečku je proto nutné přidat mezeru nebo znak nového řádku `\n`.

```
String pat = "=name= ve svém příspěvku =verb=, že...#" +
  "cond([verb := 'napsal', gen := 'm'])." +
  "cond([verb := 'napsala', gen := 'f']).";
```

Příklad 6.3: Příklad chybného zadání vzorce.

[22] *Unifikace* je taková substituce proměnných ve dvou (nebo více) prologovských termech, aby ve výsledky všechny termy byly stejné (až na názvy proměnných). Pro podrobnější vysvětlení viz [10].

Popis jazyka a slovník

Slovník je definován prologovskou procedurou `item(+Index, +Ort, +Features)`.[23] Význam jejích parametrů je popsán v tabulce 6.5. Příklad 6.4 obsahuje ukázkové slovníkové záznamy.

Název parametru	Popis
`Index`	Index ve slovníku.
`Ort`	Pravopisný zápis konkrétní formy. Jelikož jde o již konkretizovaný parametr, musí se jednat o atom. Zápis doporučujeme vždy uzavírat do uvozovek.
`Features`	Seznam *rysů* ve tvaru `Nazev==Hodnota`.

Tabulka 6.5: Popis argumentů procedury `item/3`.

```
item('go', 'jdu', [person==first, num==sg]).
item('go', 'jde', [person==third, num==sg]).
```

Příklad 6.4: Ukázkové slovníkové záznamy.

Při definování slovníku lze použít i pomocné predikáty. Součástí knihovny `nlpat` je jednoduchá morfologie českého jazyka. Přídavná jména (lze přidat i další slovní druhy) lze definovat pomocí predikátu `define_cz(+Index, +Root, +Type)`,

[23] Znaky, které předchází názvům parametrů v dokumentaci prologovských procedur, mají následující význam [10]:

`+` – vstupní (již konkretizovaný parametr),

`-` – výstupní parametr, na tomto místě může stát nekonkretizovaná proměnná,

`?` – vstupní nebo výstupní parametr.

Název parametru	Popis
`Index`	Index ve slovníku.
`Root`	Kmen slova (včetně morfologického vzoru).
`Type`	Slovní druh. Podporovány jsou dva typy: **adjh** tvrdé přídavné jméno (např. *krásný*) **adjw** měkké přídavné jméno (např. *jarní*)

Tabulka 6.6: Popis argumentů procedury `define_cz/3`.

není třeba vyjmenovávat všechny možné tvary. Popis parametrů je uveden v tabulce 6.6. Např. přídavné jméno *krásný* lze do slovníku přidat následujícím způsobem: `‘define_cz(beautiful, ’ákrsn’, adjh)‘`.

Vyhodnocování

Vyhodnocování vzorců zajišťuje metoda `String evaluate(String... params)` javovské třídy `cz.cuni.mff.ufal.volk.patterns.Pattern`. Přijímá sudý počet argumentů – střídavě názvy parametrů ze vzorce a literály nebo slovníkové indexy se znakem `@` na začátku. Pokud literál začíná znakem `@`, je třeba ho zdvojit. Příklad 6.5 ukazuje kompletní kód v Javě pro vytvoření textové zprávy.

```
// povinné importy, dál se používají pouze názvy tříd
import alice.tuprolog.InvalidTheoryException;
import cz.cuni.mff.ufal.volk.patterns.LanguageDescription;
import cz.cuni.mff.ufal.volk.patterns.Pattern;
```

```
LanguageDescription czechDesc;
FileInputStream in = new FileInputStream("pl/cz_lang.pl");
try {
  czechDesc = new LanguageDescription(in);
} finally {
  in.close();
}
Pattern pat = new Pattern(desc, "Ty jsi =b= holka.#" +
  "cond([b-->[case == nom, num == sg, gen == fem]]).");
String out = pat.evaluate("b", "@beautiful");
```

Příklad 6.5: Příklad použití modulu `nlpat`. Předpokládá se, že program má přístup k souboru `pl/cz_lang.pl` a že tento soubor obsahuje správnou definici jazyka.

Příklady použití

Příklady použití (všechny níže uvedené příklady jsou součástí testovacího balíku `cz.cuni.mff.ufal.volk.patterns.test`):

- Výběr koncovky přídavného jména podle rodu. Koncovka je pevně zadaná v kódu, slovo pochází ze slovníku.

  ```
  Toto je krásn=ending= =noun=.#
    cond([ending:='á',
      noun-->[case==nom, num==sg, gen==fem]]).
    cond([ending:='ý',
      noun-->[case==nom, num==sg, gen==masc]]).
  ```

  Kód: `pat.evaluate("noun", "@hedgehog");`

Výsledek: *Toto je krásný ježek.*

- Shoda podstatného a přídavného jména v rodě:

```
=a= =n= =v=#
cond([
  a --> [case==nom, num==sg, gen == G],
  n --> [case==nom, num==sg, gen==G],
  v --> [person==third, num==sg]
]).");
```

Kód: `pat.evaluate("a", "@beautiful", "n", "@girl", "v", "@eat");`

Výsledek: *krásná holka jí*

- Shoda podstatného jména a slovesa v rodě. V tomto příkladu se nepoužívá slovník; slova, kterými mají být nahrazeny proměnné, jsou zakódovánas ve vzorci.

```
=name= =verb=, že...#
  cond([verb := 'napsal', gen := 'm']).
  cond([verb := 'napsala', gen := 'f']).
```

Kód: `pat.evaluate("name", "Magda", "gen", "f");`

Výsledek: *Magda napsala, že...*

6.8.7 Rozpoznání jazyka – `tool_langr`

Knihovna `tool_langr` slouží jako implementace rozpoznávání jazyka psaného textu. Zakládá se na n-gramovém (viz [12]) jazykovém modelu (číslo n je parametrizovatelné) nad znaky. Trénovací data pro každý jazyk by měla být rozdě-

lena na část pro samotné trénování a část, která bude použita pro vyhlazování jednotlivých jazykových modelů.

Hlavní třídou této komponenty je `NgramLanguageRecognizer` (jakožto implementace rozhraní `LanguageRecognizer`). Příprava její instance, která umožňuje rozpoznat jazyk textu, probíhá ve dvou fázích:

1. Volání konstruktoru `NgramLanguageRecognizer(Mode mode)`. Parametr je nepovinný, rozpoznávání jazyka funguje s výchozí hodnotou `Mode.Normal`.

2. Volání metody `void train(File source, int n)`, která slouží trénování jazykových modelů; popis parametrů se nachází v tabulce 6.7. Při testování (polština, čeština, slovenština, maďarština a kašubština) jsme zjistili, že kvalita rozpoznávání je vyhovující již pro $n = 5$ (úspěšnost $93,37\%$ pro použitá testovací data). Zvětšování čísla n způsobuje významné zpomalení procesu trénování a zvětšení modelu (11MB pro $n = 6$ oproti 6MB pro $n = 5$) a nemá skoro žádný vliv na kvalitu rozpoznávání. Po nastavení $n = 6$ jsme dokonce zaznamenali mírný pokles úspěšnosti na $93,20\%$.

Hotový model lze serializovat do souboru a takto distribuovat, aniž by bylo nutné poskytovat trénovací data – to má dvě hlavní výhody:

1. Trénování může trvat delší dobu (řadově až desítky minut), takže serializace vede k výrazné časové úspoře.

2. Mají-li být trénovací data veřejně přístupná, poskytovatel musí použít pouze texty, které má právo veřejně distribuovat. Tento problém odpadá, je-li možné uživatelům dodat pouze jazykový model.

V resources balíčku `tool_langr.jar` je jako `model/slav1.nmodel` zpřístupněn serializovaný objekt, který umožňuje rozpoznávání češtiny (`cs`), slovenštiny (`sk`),

Parametr	Popis

`source`	Adresář, kde se nachází trénovací data. Jeho struktura musí vypadat takto:
	`./`
	`train/`
	soubory s trénovacími daty
	`smooth/`
	soubory s daty používanými pro vyhlazování modelů
	Každý podporovaný jazyk musí být zastoupen jedním souborem v každém z podadresářů; jeho názvem je ISO-639 kód jazyka bez přípony, např. `cs`, `sk`, `pl`.

`n`	Úroveň (maximální délka n-gramů, pro které se počítá pravděpodobnost) modelů.

Tabulka 6.7: Popis parametrů metody `NgramLanguageRecognizer.train`.

polštiny (`pl`), maďarštiny (`hu`) a kašubštiny (`csb`). Podporuje rozpoznávání jazyků pro texty s diakritikou i bez ní.[24]

Zatím je v resources jenom jeden model, lze nicméně serializovat a přidat další modely. Jednoduchý přístup k serializovaným modelům zajišťuje třída `ResourceLanguageRecognizer`. Způsob načítání modelů pro rozpoznávání jazyka je znázorněn v příkladech 6.6 a 6.7.

```
LanguageRecognizer<Text> rec =
```

[24]Jako trénovací data byly použity texty z Wikipedie v příslušných jazycích. Jazykové modely jsou pětigramové.

```
  new ResourceLanguageRecognizer("slav1.nmodel");
```

Příklad 6.6: Příklad kódu pro načtení serializovaného modelu z resources balíčku `tool_langr.jar`.

```
<bean id="volk.langrec" class="cz.cuni.mff.ufal.volk.langr.
    ResourceLanguageRecognizer">
  <constructor-arg value="slav1.nmodel" />
</bean>
```

Příklad 6.7: Příklad načtení serializovaného modelu z balíčku `tool_langr.jar` s použitím Spring Frameworku. Nejedná se o kompletní konfiguraci, pouze o část, která zodpovídá za načtení objektu.

Kapitola 7

Závěr

V rámci této práce jste připravili návrh obecného API pro ovládání robotů v přirozeném jazyce a jeho ukázkovou implementaci pomocí programovacích jazyků Java a Prolog a robotického králíka Nabaztag:tag. Toto API specifikuje komunikaci mezi robotem a serverem, mezi uživatelem a serverem a umožňuje zpracování přirozeného jazyka, zpracování získané informace (např. generování hlasu) a napojení robota na externí služby. Byla implementována oboustranná komunikace mezi agentem (robotickým králíkem Nabaztag:tag) a ovladačem (komunikačním serverem `volk`). Implementovali jsme komunikační protokoly, příjem zvukových záznamů na serveru, posílání robotovi příkazů (pohyb, světelné efekty, digitální audio). Komunikaci s uživatelem v přirozeném jazyce zajišťují: automatické vytváření zvukových záznamů na základě textových zpráv, generování textových zpráv v přirozeném jazyce na základě vzorců, jednoduchý interpret příkazů pro robota Nabaztag:tag v přirozeném jazyce (zatím pouze čeština) a komponenta, která umožňuje rozpoznat jazyk textu v přirozeném jazyce. Ukázková implementace obsahuje i příklad napojení robota na služby třetích stran (čtečka RSS

zdrojů).

Ovladač `volk`, který jsme vytvořili, umožňuje komunikovat s robotem čistě v rámci lokální Wi-Fi sítě, není přitom vyžadováno připojení k Internetu[1]. Díky tomu je aplikace úplně nezávislá na externích serverech. Navíc se oproti oficiálnímu serveru výrazně zmenšila doba odezvy králíka, která se teď odvíjí pouze od frekvence odesílání dotazů ping králíkem (viz popis konfigurace ovladače v příloze B.1 na stránce 104) a času nutného pro zpracování příkazu.[2]

Zpřístupnuli jsme předpřipravenou konfiguraci serveru (viz kapitola B.2). Ta umožňuje mj.:

- ovládání robota pomocí jednoduchých textových příkazů,
- generování mluvené řeči v českém jazyce,
- stahování a čtení RSS zpráv.

Nejaktuálnější verze zdrojových kódů jsou dostupné ke stažení na stránce `http://code.google.com/p/volk/`. Lze je stáhnout i z SVN repozitáře pomocí následujícího příkazu (anonymní přístup jenom pro čtení):

```
svn checkout http://volk.googlecode.com/svn/trunk/
   volk-read-only
```

[1] Některé služby, např. čtení RSS zpráv, můžou vyžadovat aktivní internetové připojení.

[2] Výrazné zlepšení nastává ve srovnání s oficiálním HTTP serverem. Proti komunikaci s novější verzí oficiálního serveru, která používala protokol XMPP, není zrychlení tak velké, ale přesto zřetelné (viz kapitola 3).

7.1 Další možnosti vývoje

Níže stručně popisujeme další možné směry vývoje samotného API, jeho implementace pro robotického králíka Nabaztag:tag i jednotlivých zásuvných modulů.

- Zajištění kompatibility ovladače s jinými verzemi králíka (starší Nabaztag, novější Karotz) – a přitom zohlednit omezení starší generace (např. možnost přehrávat maximálně 45 s zvuku), popř. i zajistit podporu novinek v robotovi Karotz (vnitřní úložiště, USB port, webová kamera), což ale může vyžadovat nahrazení HTTP serveru XMPP serverem.

- Sjednocení použitého enginu Prologu pro všechny komponenty, které vyžadují Prolog. Jednalo by se pravděpodobně o použití SWI Prologu. TuProlog podle všeho nebude v dohlédné době natolik pokročilý, aby mohl nahradit SWI.

- Vývoj pluginů pro automatické rozpoznávání mluvené řeči a hlasové ovládání; stávající verze umožňuje napojení takových pluginů, konkrétní pluginy nicméně nejsou zatím implementovány.

- Implementace pokročilejších pluginů pro porozumění textům v přirozeném jazyce.

- Integrace parsovacích systémů (TectoMT, MaltParser, MST, Collins).

- Vývoj pluginů pro porozumění textům v dalších jazycích.

- Analýza vhodnosti serveru nedávno uvolněného výrobcem pro domácí používání.

- Další vývoj základních datových typů, které reprezentují sdělení v přirozeném jazyce.
- Použití aspektově orientovaného programování pro rozšíření logování.
- Použití standardního nástroje pro sestavování aplikací (např. Apache Ant[3] nebo Apache Maven[4]).

[3] `http://ant.apache.org/`
[4] `http://maven.apache.org`

Literatura

[1] About Wolfram|Alpha *Making the world's knowledge computable* [online]. Publikováno 2010 [cit. 2011-11-08]. Dostupné z: `http://www.wolframalpha.com/about.html`.

[2] COLMERAUER, A., ROUSSEL, P. *The birth of Prolog*. Publikováno 1992 [cit. 2011-10-14]. Dostupné z: `http://alain.colmerauer.free.fr/alcol/ArchivesPublications/HistoireProlog/19november92.pdf`.

[3] Encyclopædia Britannica Online *API*. Publikováno 2011 [cit. 2011-11-08]. Dostupné z: `http://www.britannica.com/EBchecked/topic/1472947/API`.

[4] Eurofighter *Direct Voice Input* [online]. Publikováno 2010 [cit. 2011-08-14]. Dostupné z: `http://www.eurofighter.com/capabilities/technology/voice-throttle-stick/direct-voice-input.html`.

[5] FOWLER, M.: *Architektura korporativnych programmnych prilozhenij*, Moskva: Izdatel'skij dom Williams, 2006, s. 43 – 50.

[6] HAJIČOVÁ, E., PANEVOVÁ, J., SGALL, P. *Úvod do teoretické a počítačové lingvistiky*, Praha: sv. I. Karolinum, 2002.

[7] *Changelog for Opera 8.0 for Windows* [online]. Publikováno 2005 [cit. 2011-11-08]. Dostupné z: HTTP://www.opera.com/docs/changelogs/windows/800/.

[8] JOHNSON, R. et al. *Reference Documentation*. Publikováno 2010 [cit. 2011-11-08]. Dostupné z: http://static.springsource.org/spring/docs/3.0.x/spring-framework-reference/pdf/spring-framework-reference.pdf.

[9] *jNabServer – Control your bunny* [online]. Publikováno 2008 [cit. 2011-09-26]. Dostupné z: http://www.cs.uta.fi/hci/spi/jnabserver/#protocol.

[10] KRYL, R.: *Úvod do programovacího jazyka Prolog*. Ver. 3.03 [cit. 2011-12-05]. Dostupné z: http://ksvi.mff.cuni.cz/~kryl/prolog.pdf.

[11] Mac OS X Developer Library *Introduction to AppleScript Overview* [online]. Publikováno 2007 [cit. 2010-08-04]. Dostupné z: http://developer.apple.com/applescript/.

[12] MANNING, C. D., SCHÜTZE, H. (2003) *Foundations of Statistical Natural Language Processing*, 6. vyd. Massachusetts Institute of Technology, 2003. S. 191 – 197.

[13] MSDN *Globalization Step-by-Step* [online]. Publikováno 2011 [cit. 2011-09-07]. Dostupné z: http://msdn.microsoft.com/en-us/goglobal/bb688112.

[14] PECINOVSKÝ, R. *Návrhové vzory*, Brno: Computer Press a.s., 2007.

[15] The Epos Speech System *Text-To-Speech Control Protocol (version 0)* [online]. [cit. 2011-09-30]. Dostupné z: `http://epos.ure.cas.cz/documentation/epos-5.html`.

[16] Ubiquity *An experimental interface based on natural language input* [online]. Publikováno 2010 [cit. 2011-09-15]. Dostupné z: `http://mozillalabs.com/ubiquity/`.

[17] WEIZENBAUM, J. (1966) *ELIZA — A Computer Program For the Study of Natural Language Communication Between Man And Machine*, Communications of the ACM, vol. **9**, s. 37–42.

[18] Wikipedia. *Nabaztag*. [cit. 2011-07-27]. Dostupné z: `http://en.wikipedia.org/wiki/Nabaztag`.

[19] *What can I do with Speech Recognition?* [online]. Publikováno 2011 [cit. 2011-08-04]. Dostupné z: `http://windows.microsoft.com/en-US/windows7/What-can-I-do-with-Speech-Recognition`.

[20] WINOGRAD, T. *Procedures as a Representation for Data in a Computer Program for Understanding Natural Language* Publikováno 1971 [cit. 2011-11-08]. Dostupné z: `http://dspace.mit.edu/handle/1721.1/7095`.

Příloha A

Obsah přiloženého CD

- `/`
  - `thesis.pdf` – text diplomové práce
  - `src/` – zdrojové kódy naší implementace API
  - `bin/` – spustitelná distribuce
  - `javadoc/` – vyexportovaná programátorská dokumentace (pouze pro javovské zdrojové kódy)
  - `config/` – konfigurační soubory
    - `spring/` – konfigurace aplikace pro různé operační systémy
      - `volkapp.config.xml_win32` – výchozí konfigurace pro 32-bitové systémy Windows
      - `volkapp.config.xml_win64` – výchozí konfigurace pro 64-bitové systémy Windows
      - `volkapp.config.xml_other` – vzor konfigurace pro systém, ve kterém jsou pomocné aplikace nainstalovány v nestandardních umístěních
    - `log4j/` – předdefinované konfigurace logování

└ `volkapp.log4j.txt*` – soubory s předdefinovanými konfiguracemi, podrobnosti viz tabulka B.1

Příloha B

Nastavení a použití robotického králíka Nabaztag:tag

B.1 Konfigurace robota

Výchozí konfigurace robota umožňovala připojení pouze k firemním serverům Violet, proto je pro připojení k jinému serveru potřeba výchozí konfiguraci změnit.

1. Zapojte robota do elektřiny se stisknutým tlačítkem na jeho hlavě. Až se diody na králíkovi rozsvítí tmavě modře, můžete tlačítko pustit. Robot vytvoří nezabezpečenou Wi-Fi síť `NabaztagXX`, kde `XX` jsou dvě poslední číslice sériového čísla robota.

2. Připojte počítač k Wi-Fi síti vytvořené robotem.

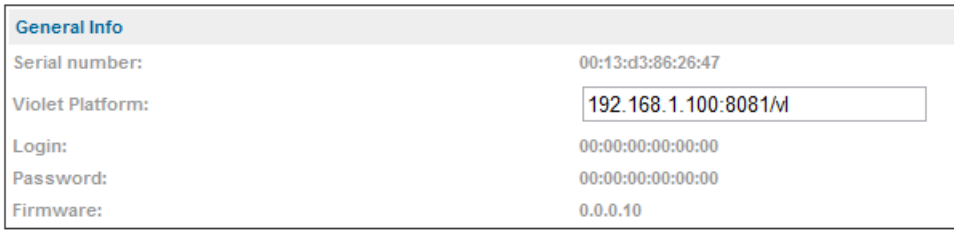
3. V libobolném internetovém prohlížeči otevřete administraci robota, která je dostupná na adrese `192.168.0.1`.

4. Zadejte jméno (SSID) Wi-Fi sítě, ve které bude dostupný server – ovladač a ke které se musí připojit robot. Nakonfigurujte šifrování.

5. Klikněte na *Advanced configuration*.

6. Nastavte další údaje pro připojení k síti, je-li třeba.

7. Nastavit IP adresu a port serveru (není-li uveden port, předpokládá se standardní pro HTTP port 80), viz obrázek B.1. Za adresu dopište `/v1`.



Obrázek B.1: Nastavení adresy serveru, se kterým bude Nabaztag:tag komunikovat.

8. Klikněte na tlačítko *Update and start*. Nabaztag:tag se připojí k určené síti a začne komunikovat se serverem.

Během pokusu o připojení k Wi-Fi síti a k serveru indikují diody na robotovi aktuálně probíhající činnost (tyto informace můžou být nápomocné v případě řešení problémů s připojením):

1. *Všechny diody svítí oranžově*: čeká se na pokus o připojení k Wi-Fi síti.

2. Dioda `01` svítí zeleně, `02` bliká oranžově, ostatní svítí oranžově – pokus o připojení k Wi-Fi síti.

3. Všechny tři diody na břiše králíka (`01`, `02`, `03`) svítí zeleně, diody na nose (`00`) a na spodní straně (`04`) blikají oranžově: připojení k Wi-Fi síti bylo

navázáno. Odesílá se dotaz na bootcode (`GET http://%s/vl/bc.jsp`, viz kapitola 3.3).

4. Všechny diody se na chvíli rozsvítí zeleně, potom králík naklopí uši a diody přestanou svítit – stažení bootcodu se zdařilo, Nabaztag:tag komunikuje se serverem.

B.2 Konfigurace serveru

Obsah složky `bin/` z CD zkopírujte do lokálního adresáře. Dále vytvořte konfigurační soubor `volkapp.config.xml`. Chcete-li použít jeden ze souborů s výchozí konfigurací, před spuštěním aplikace ho přejmenujte na `volkapp.config.xml`.

Výchozí konfigurace

Výchozí konfigurace serveru (dostupná na CD) umožňuje mj.:

- ovládání robota pomocí jednoduchých textových příkazů,
- generování mluvené řeči v českém jazyce,
- stahování a čtení RSS zpráv.

Pro zprovoznění výchozí serveru s výchozí konfigurací je potřeba dále nainstalovat následující software:

- Epos, verze 2.4.83 (`http://http://epos.ure.cas.cz/`). Aplikace nefunguje s verzí 2.5 a novějšími.
- SWI Prolog, verze 5.10 nebo novější (`http://www.swi-prolog.org/`). Po instalaci rozšiřte systémovou proměnnou `PATH` o cestu adresáře, ve kterém

se nachází knihovna `jpl.dll` nebo `libjpl.so` (podle operačního systému). Tento krok je nutný, aby aplikace napsané v Javě mohly používat knihovnu JPL.

- Sound eXchange, verze 14.3.2 nebo novější (`http://sox.sourceforge.net/`).

Jsou-li v operačních systémech Windows (verze XP, Vista nebo 7) výše uvedené programy nainstalovány ve výchozích umístěních, lze použít beze změn jeden ze dvou předem připravených konfiguračních souborů:

`volkapp.config.xml_win32` pro 32-bitové systémy,

`volkapp.config.xml_win64` pro 64-bitové systémy.

Soubor `volkapp.config.xml_other` je určen pro jiné operační systémy nebo nestandardní instalace. Je třeba ho použít s následujícími změnami:

- řetězec `EPOS_PATH` nahradit plnou cestou spustitelného souboru `epos` nebo `epos.exe`
- řetězec `SOX_PATH` nahradit plnou cestou spustitelného souboru `sox` nebo `sox.exe`

Vlastní konfigurace

Můžete vytvořit i libovolnou vlastní konfiguraci kompatibilní se Spring Frameworkem.

B.3 Konfigurace logování

Nastavení logování se vždy načítá ze souboru `volkapp.log4j.txt` (za předpokladu, že tento soubor existuje; viz 6.3). Vytvořili jsme několik konfigurací logování, jsou popsané v tabulce B.1. Uživatel může použít i vlastní konfigurační soubory.

Název souboru	Standardní výstup	Soubor `volkapp.log`
`volkapp.log4j.txt`	`WARN`	`TRACE`
`volkapp.log4j.txt_info`	`WARN`	`INFO`
`volkapp.log4j.txt_chatty`	`INFO`	`TRACE`
`volkapp.log4j.txt_all`	`TRACE`	`TRACE`

Tabulka B.1: Předdefinované konfigurace logování. Všechny nastavují logování na standardní výstup a do souboru `volkapp.log`.

B.4 Spuštění serveru

Server lze v adresáři, kde se nachází soubor `volkapp.jar`, spustit z příkazové řádky pomocí následujícího příkazu:

```
java -classpath "volkapp.jar;lib/*" cz.cuni.mff.ufal.volkapp.Main
```

Nepoužívá-li konzola kódování UTF-8, je navíc třeba nastavit výchozí kódování znaků parametrem `"-Dfile.encoding=ENC"` (včetně uvozovek), kde `ENC` představuje kódování konzoly (např. `cp852` pro česká Windows).

B.5 Nahrávání zvuku

Aby Nabaztag:tag začal zaznamenávat zvuk, je třeba cca 2s podržet tlačítko na jeho hlavě. Jakmile začne pořízování záznamu, dioda na nose králíka se rozsvítí červeně. Nechte tlačítko stisknuté po celou dobu pořizování záznamu.